

基于几何机理与协同调度的无人机集群烟幕投放策略优化

摘要

现代军事防御体系中，软杀伤干扰技术逐渐成为应对精确制导武器威胁的重要手段之一。烟幕干扰作为无源防御手段，通过化学燃烧或爆炸形成气溶胶云团，在目标前方构建遮蔽屏障，可有效干扰多波段侦察与制导，具有响应快、覆盖广、效费比高等优势。随着技术进步，烟幕干扰弹已能实现定点投放与时序控制。无人机凭借机动与定位优势，成为烟幕布设的重要平台，可在来袭武器与目标间快速形成遮蔽。为实现更优干扰效果，需合理设计烟幕干扰弹投放与起爆策略以最大化遮蔽时间。本文研究多无人机协同投放的时空优化模型，旨在通过几何构型设计与任务调度协同，显著增强集群烟幕干扰效能。

针对问题 1，提出了“双模型互验”求解框架，突破传统单一方法局限。通过建立导弹—烟幕—目标**动态几何模型**，借助**参考系变换**，将问题转化为静态烟幕球与运动导弹轨迹的精确求交，得出关键时间点的解析表达式；同时设计**高精度时间离散化验证算法**，动态检测导弹位置与烟幕球的包含关系，独立计算遮蔽时长。结果表明，两种方法所得有效遮蔽时长**高度一致**（收敛于 1.40 s），相互验证了几何解析的**严谨性**与**算法的可行性**。

针对问题 2，提出了**边界引导的粒子群优化框架**（BGE-PSO），每个粒子通过位置向量 $(\theta, v, t_{\text{drop}}, t_{\text{burst}})$ 编码一组完整的烟幕干扰弹投放策略。引入**动态边界奖励项**后，算法在**约束边界附近的搜索能力**显著增益，传统算法难以处理**极值点优化**的问题得以有效解决。经 100 次迭代，得到全局最优解，最终实现 **4.80 s** 的有效遮蔽时间。灵敏度分析则进一步表明，该算法在参数边界区域展现了**优异的收敛性和稳定性**，为实际战术决策提供了可靠依据。

针对问题 3，构建了**基于方向对冲与权重优化的多弹协同遮蔽策略**。通过调整无人机飞行方向（与导弹 x 轴速度分量相反）并采用立即引爆策略，有效增强多弹遮蔽连续性；创新性引入权重机制优先保障后投弹体的遮蔽贡献，建立以导弹轨迹与各烟幕球心距离加权和最小化为目标的函数。经约束优化求解，得到三枚烟幕弹最优爆炸点坐标，最终实现 **6.51 s** 的最大遮蔽时长，验证了多弹协同策略在提升单机对抗效能方面的有效性。

针对问题 4，构建了**多无人机—单导弹协同干扰模型**。通过引入威胁权重系数，我们量化了不同导弹的战术价值，并建立以加权遮蔽时间为目标的优化函数。同时，采用**改进鲸鱼优化算法**（WOA）求解，利用模拟鲸鱼包围捕食和随机搜索行为，高效处理了**多约束条件下的资源分配问题**。结果表明，该算法能有效协调 3 架无人机对 1 枚导弹的干扰任务，避免空间冲突的同时，实现了**整体遮蔽效能最大化**，为无人机集群协同作战提供了可行的决策框架。

针对问题 5，在问题四构建的多无人机—单导弹协同框架基础上，引入价值系数矩阵与多因子任务分配算法，实现了**多无人机—多导弹协同遮蔽**的智能决策。通过结构化价值系数量化每架无人机对每枚导弹的遮蔽潜力，并构建**带容量约束的分配模型**，将复杂多对多问题分解为多个多无人机—单导弹子问题并行优化。进一步采用距离衰减型覆盖函数与混合优化算法（WOA+SLSQP），在严格弹量约束下协同优化投放点与时序。最终求解获得最大遮蔽时长 14.7325 s，验证了所提方法在复杂多任务场景下的有效性，凸显了时空配置与协同分配对系统作战效能的关键作用。

关键词：烟幕干扰；有效遮蔽；粒子群算法；鲸鱼优化算法；几何建模

一、问题重述

1.1 研究背景

在现代军事防御体系中，软杀伤干扰技术因其效费比高、响应迅速等特点，逐渐成为应对精确制导武器威胁的重要手段之一。烟幕干扰弹通过特定化学反应生成气溶胶云团，可在目标区域形成光学遮蔽区间，有效干扰敌方导引头的探测与跟踪过程。无人机系统凭借其强机动性与精准定位能力，为烟幕的高效、灵活布设提供了重要支撑。

随着导弹武器速度不断提升、攻击模式日趋复杂，单一固定式干扰手段恐难应对多批次、多方向的饱和攻击。无人机协同实施烟幕遮蔽任务，具备快速响应、空间灵活与智能协同的优势，对提升要地防空生存能力至关重要。因此，开展面向多约束条件下烟幕弹投放策略的优化研究，不仅有助深化干扰资源配置理论，也对提升实际作战效能具有迫切需求。

1.2 问题提出

设定一个三维直角坐标系：以虚假目标所在位置作为坐标原点 $O(0,0,0)$ ，地面对应 xy 平面， z 轴正向竖直朝上。真实目标为圆柱体，其底部圆面中心坐标为 $(0,200,0)$ ，圆柱半径为 7 m ，总高度为 10 m 。来袭导弹以 300 m/s 的速度匀速飞行，且其航向始终对准虚假目标原点。在初始时刻，三枚导弹的空间位置分别为：M1 $(20000,0,2000)$ ，M2 $(19000,600,2100)$ ，M3 $(18000,-600,1900)$ 。五架无人机的起始坐标分别为：FY1 $(17800,0,1800)$ ，FY2 $(12000,1400,1400)$ ，FY3 $(6000,-3000,700)$ ，FY4 $(11000,2000,1800)$ ，FY5 $(13000,-2000,1300)$ 。

基于上述建模框架和物理约束，本研究需要解决以下五个具体问题：

问题 1，聚焦于单一作战单元的最优效能评估。 给定无人机 FY1 以 120 m/s 的恒定速率向虚拟靶标方向航行，在接收指令 1.5 s 后实施弹药布设，并于 3.6 s 后触发引爆装置。需精确计算在此预设参数条件下，烟雾云团对导弹 M1 的有效遮蔽时间区间。

问题 2，在问题一基础上演进为单目标优化问题。 仍维持单一无人机对单一导弹的对抗架构，但需自主优化四个关键决策变量：飞行航向角、航行速率、弹药释放时空坐标与引爆时间点。优化目标函数为遮蔽时间最大化。

问题 3，将系统复杂度提升至多弹药协同层次。 要求基于 FY1 平台连续布设三枚干扰弹药，通过对 M1 实施序列化干扰来增强遮蔽效果。需统筹设计包括布设时间序列、空间分布模式，与起爆时序控制的完整策略体系，同时满足相邻弹药投放最小时间间隔的技术约束。

问题 4，进一步扩展为多平台协同作战模式。 应协调 FY1、FY2、FY3 三架无人机各携带一枚弹药，共同构建对 M1 的复合干扰网络。要解决多平台协同路径规划、时空同步优化等关键问题，属于典型的多智能体系统协同优化范畴。

问题 5，构成全局性综合优化问题。 要求调度全部五架无人机（各平台最多携带三枚弹药），同步对抗三枚来袭导弹的威胁。需要建立大规模混合整数规划模型，解决包括任务分配、路径规划、时序协调等多层次优化问题，最终形成全局最优的作战策略。

二、问题分析

2.1 问题 1 的分析

问题一要求计算在给定策略下，单枚烟幕干扰弹对导弹 M1 的有效遮蔽时长，核心难点在于如何准确描述导弹、下沉烟幕云团及圆柱形目标三者间随时间演变的复杂几何关系，并据此判断有效遮蔽的时间窗口。我们尝试提出“双模型互验”的求解思路：首先建立严格的运动学与几何分析模型，通过参考系变换将导弹轨迹与静态烟幕球体求交，解析计算关键遮蔽时间点，以得出遮蔽总时长；进而设计高精度离散时间遍历算法，动态检测导弹轨迹点与烟幕球体的包含关系，独立计算结果并回溯验证。两种方法相互印证，有望大幅减少传统单一方法可能存在的模型误差或数值偏差，为后续复杂场景建模提供可验证的理论基础。

2.2 问题 2 的分析

问题二本质是高维非线性优化问题，关键在于协调无人机运动、弹体下落与烟幕扩散的时空关系，以确保烟幕在导弹抵达目标前持续遮蔽其视线路径。遮蔽效果高度依赖飞行参数与起爆时序的协同优化，也需严格满足速度、高度和时间顺序等多重物理约束。由于最优解常分布于参数边界附近，解题难度进一步加大。因此，我们可以建立以遮蔽时间为目标的优化模型，引入边界奖励机制处理约束与边界寻优问题；同时可采用改进粒子群算法（PSO）进行高效求解，从而实现对关键决策变量的全局优化，尝试突破传统方法在复杂战术决策上的瓶颈。

2.3 问题 3 的分析

问题三研究单无人机投放三枚烟幕弹对抗单导弹的协同优化问题，核心在于协调多弹时空分布与导弹运动间的匹配关系。我们尝试建立基于相对运动与弹道匹配的多弹协同模型，通过调整无人机飞行方向与立即引爆策略增强遮蔽连续性，并引入权重机制区分不同烟幕弹的遮蔽贡献。模型以导弹轨迹与各烟幕球心距离加权和最小化为目标，优化投放时序与空间位置，在满足速度、间隔等约束下实现最大遮蔽时长。

2.4 问题 4 的分析

问题四针对多无人机协同对抗单导弹的场景，要求优化烟幕弹投放策略，实现全域最大化遮蔽。这属非线性、多约束协同决策问题，难点在于有限资源下的时空分配与约束满足。我们可提出一种基于遮蔽贡献度的协同决策模型，尝试突破传统单目标优化或加权求和的思想：引入导弹威胁权重区分目标价值，以加权总遮蔽时间为优化目标，构建适应度函数。针对复杂约束与解空间，也可采用鲸鱼优化算法（WOA）进行求解，兼顾全局探索与局部开发，将有效提升解的质量与搜索效率，解决多机协同中的资源分配与冲突规避问题。

2.5 问题 5 的分析

问题五在问题四的多无人机协同框架基础上，进一步研究多导弹威胁下带有严格弹量约束的协同遮蔽问题。与问题四相比，我们可进一步提出基于价值系数矩阵的任务分配机制，将复杂的多对多协同问题分解为多个多对一子问题并行求解；新增每架无人机

最多投放 3 枚烟幕弹的容量约束，并采用距离衰减型覆盖函数提升遮蔽建模精度。通过改进的混合优化算法（WOA-SLSQP）协同优化资源分配与时空策略，我们有望最终实现 5 架无人机对抗 3 枚导弹场景下的整体遮蔽效能最大化，以大幅提升模型的实战能力与决策可靠性。

三、模型假设

为全力保障模型合理性与可操作性，我们在充分理解具体问题情境与核心物理过程的基础上，提出如下假设：

- （1）将无人机、导弹视为质点。
- （2）忽略空气阻力对干扰弹及烟幕云团运动的影响。
- （3）假设烟幕云团在起爆瞬间形成，且为理想球体，半径恒为 10 m。
- （4）假设烟幕云团以匀速直线方式下沉，且内部浓度均匀。
- （5）假设导弹作匀速直线运动，且飞行方向始终精确指向假目标。
- （6）假设雷达所提供的目标初始位置完全准确，且指令传输与无人机响应均无延迟。
- （7）假设不同烟幕云团的遮蔽效果互相独立，不存在叠加或干扰。
- （8）假设无人机具备充足续航能力，飞行过程中无需考虑燃料限制。
- （9）忽略所有参数（如速度、坐标等）测量与控制中的随机误差。

四、符号说明

符号	含义	单位
τ	有效遮挡时间	s
t_{drop}	投放间隔时间	s
t_{burst}	爆炸间隔时间	s
v_{ij}	价值系数	/
$Cost_{ij}$	部署成本	/

五、模型建立与求解

5.1 通用模型的建立与求解

本研究针对烟幕弹投放策略展开探讨，所涉情形较综合，五个问题均需借助若干关键数学模型进行分析。详细分析具体问题前，我们首先建立并求解了以下最普适的模型。

5.1.1 无人机运动与烟幕弹投放点确定

无人机 FY1 的初始位置为 $P_0 = (17800, 0, 1800)$ 。受领任务后，无人机以速度 $v_{FY1} = 120 \text{ m/s}$ 沿水平指向原点的方向飞行，无人机在任意时刻 t 的坐标为：

$$(17800 - 120t, 0, 1800) \quad (1-1)$$

于 $t = 1.5 \text{ s}$ 时投放烟幕弹，投放点坐标 P_1 即为该时刻无人机的位置：

$$P_1 (17620, 0, 1800) \quad (1-2)$$

5.1.2 烟幕弹运动与起爆点确定

烟幕弹脱离无人机后，其水平初速度与无人机速度相同，竖直初速度为零，仅在重力加速度 $g = 10 \text{ m/s}^2$ 作用下作平抛运动。从投放时刻起，经时延 $t = 3.6 \text{ s}$ 后起爆。起爆点坐标 P_2 为

$$(17188, 0, 1735.2 - 3t) \quad (1-3)$$

5.1.3 烟幕云团运动模型

烟幕弹于 $t = 5.1 \text{ s}$ 时刻起爆，瞬时形成半径 $r = 10 \text{ m}$ 的球状云团。随后，云团中心以速度 $v_{cloud} = 3 \text{ m/s}$ 匀速下沉。因此，从起爆时刻起算，云团中心在任意时刻 t 的位置为：

$$(17188, 0, 1735.2 - 3t) \quad (1-4)$$

5.1.4 导弹运动模型

导弹 M1 初始位于 $P_2(20000, 0, 2000)$ ，以速度 $v_1 = 300 \text{ m/s}$ 直指假目标原点 $(0,0,0)$ 。因其水平分量为 298.51 m/s ，垂直分量为 29.85 m/s ，故导弹在时刻 t 的位置为：

$$(20000 - 298.51t, 0, 2000 - 29.85t) \quad (1-5)$$

进而，从起爆时刻起算，导弹在任意时刻 t 的位置为：

$$(18477.6 - 298.51t, 0, 1847.76 - 29.85t) \quad (1-6)$$

无人机 FY1 投放烟雾弹干扰导弹 M1 识别真目标的三维空间图见图 1-1。

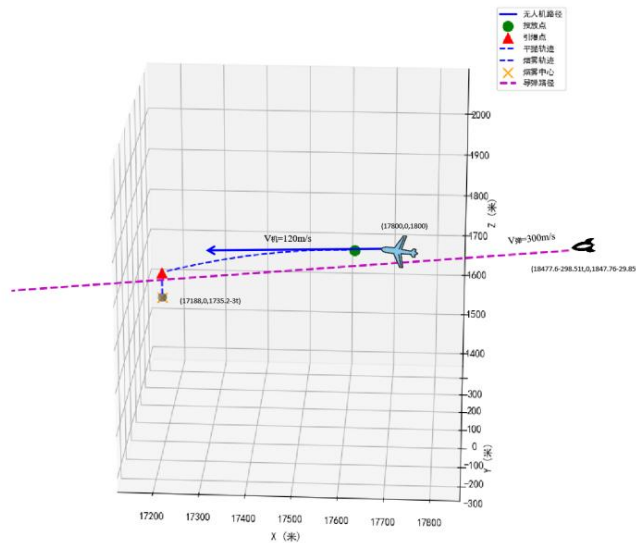


图 1-1 无人机FY1 投放烟雾弹干扰导弹识别真目标的三维空间

5.2 问题 1 模型建立与求解

针对问题 1，需计算在给定策略下单枚烟雾干扰弹对导弹 M1 的有效遮蔽时长。本问旨在验证基础模型的正确性，其核心是在三维空间中动态描述导弹、烟雾云团及保护目标间的几何关系，并基于此判断有效遮蔽的时间窗口。以下将严格按照时间发展顺序，依次建立无人机运动、烟雾弹抛体运动、烟雾云团下沉及导弹运动等数学模型，并给出有效遮蔽的判定方法。

由此，如下我们先向大家展示几何法的详细建模与计算思路。

5.2.1 相对运动模型

烟雾弹起爆瞬间，将参考系切换至以烟雾弹为原点（如图 1-2）。该参考系中，导弹有一竖直向上速度 $v_z = 3 \text{ m/s}$ ，由矢量合成的平行四边形法则，我们可以确定在此参考系下导弹速度 $v = 299.72 \text{ m/s}$ 。

计算表明，导弹在该参考系中的运动轨迹穿过以爆炸点为球心、半径为 10 米的烟雾区域。由于烟雾在该参考系中呈静止球状，易得出导弹与烟雾的相遇过程：导弹逐渐接近并进入烟雾，之后穿出，烟雾因此失去对目标的遮蔽作用。

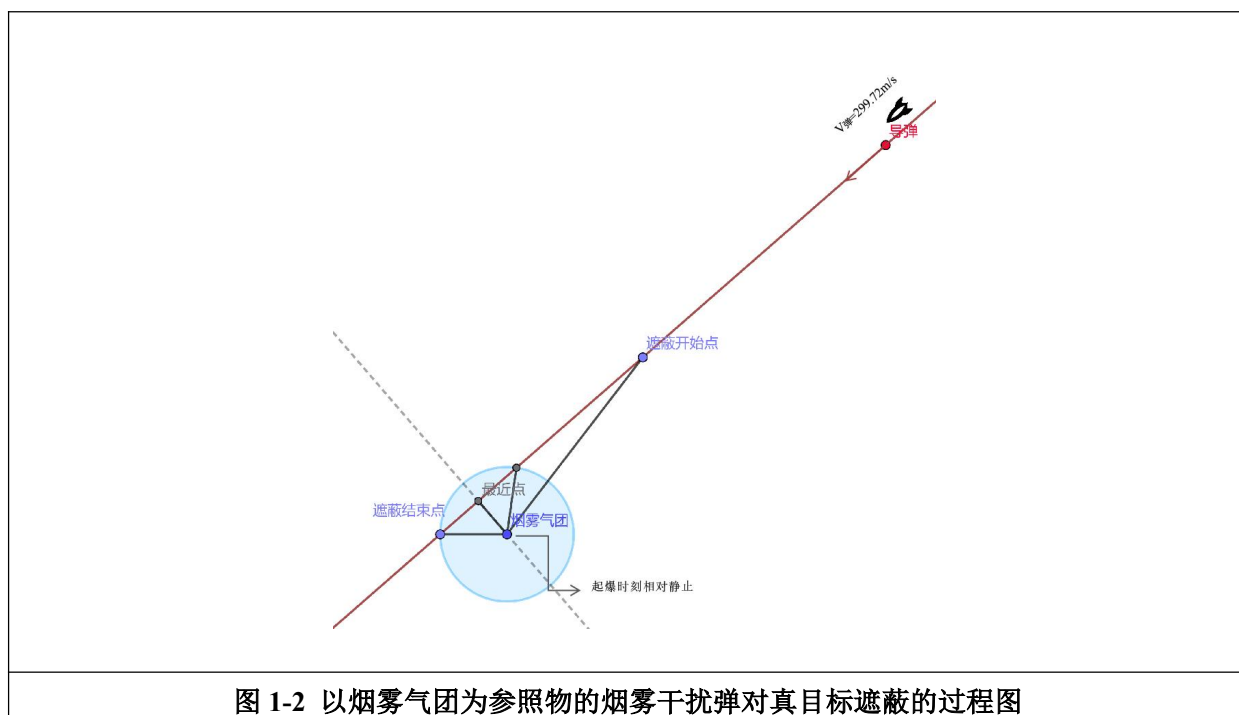


图 1-2 以烟雾气团为参照物的烟雾干扰弹对真目标遮蔽的过程图

5.2.2 完全遮蔽区间的建模分析

对于导弹的运动轨迹，分成三段进行讨论：

(1) 从烟雾弹爆炸，至导弹抵达烟雾弹完全遮蔽目标的位置

此阶段烟雾距离导弹较远，未能形成完全遮蔽，故不计入有效遮蔽时间。

(2) 从导弹刚进入完全遮蔽区域至其穿出烟雾

该阶段可进一步分为两段：导弹持续接近烟雾时，存在某一时刻起烟雾开始完全遮蔽目标，此后烟雾覆盖范围逐渐扩大，遮蔽持续生效；导弹进入烟雾后，因全方位视线阻挡，遮蔽作用仍然保持。

(3) 导弹穿出烟雾后

烟雾不再对目标具遮蔽作用，不属于有效遮蔽时间范围，与第一阶段一致。

5.2.3 遮蔽时间分析

基于前述分析，全过程中遮蔽状态由不完全逐渐转变为完全，再恢复为不完全。由于时间维度在不同参考系中不变，可在以烟雾弹为原点的参考系中，通过导弹运动轨迹函数，计算其与球心的距离，进而得到从起爆时刻至导弹轨迹与球体较远交点间的路径长度。结合该参考系中导弹的速度，可求出总时间 $t_{\text{总}}$ 。在原参考系中，该时间对应描述的是从起爆至导弹穿出烟雾的全程。

因此，我们仅需计算第一阶段未完全遮蔽的时间 t_1 ，用总时间减去该段时长即得完全遮蔽时间，表达式如下：

$$\tau = t_{\text{总}} - t_1 \quad (2-1)$$

5.2.4 第一阶段未完全遮蔽时间的计算

(1) 目标物分析

根据光直线传播原理，对于某一圆柱体而言，仅当某一平面与圆柱体相切时，整个圆柱体才位于该平面同一侧；若不相切，则平面将圆柱体分为两部分，其中一部分暴露于导弹视线范围内，无法实现完全遮蔽。（见图 2-1）

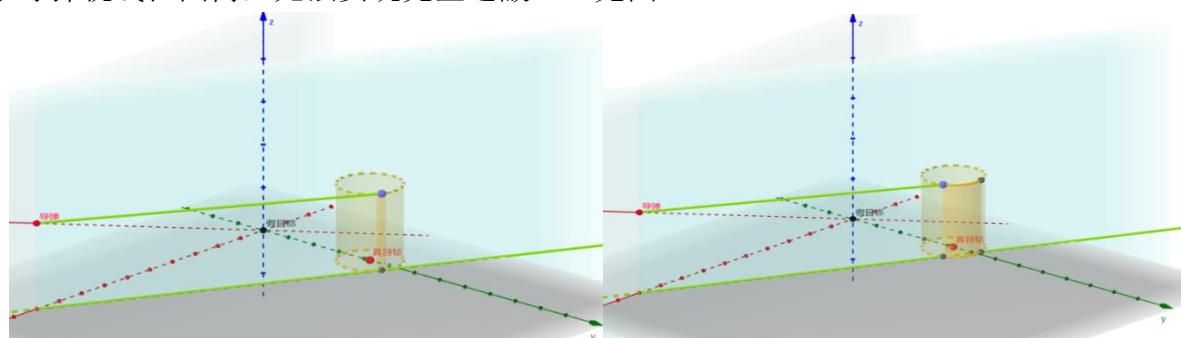


图 2-1 完全遮蔽真目标三维视线几何图

(2) 导弹位置分析

已知导弹位于 xOz 平面内，从导弹位置引出两个与圆柱体相切的平面。以导弹至圆柱体方向为正方向，将靠近圆柱体的切面称为内切面，另一侧为外切面。两切面与圆柱体相交形成两条切线段，其端点连接构成一矩形区域，称为有效阴影区域。该区域会随导弹位置变化而动态改变。（见图 2-2）



图 2-2 任意导弹位置下形成的有效阴影区域示意图

(3) 烟雾弹作用分析

烟雾弹同样于 xOz 平面释放，故仅有一个半球的烟雾对导弹产生干扰，只需考虑外切面情况。若外切面所对应的半侧有效阴影区域被遮蔽，则内切面半侧必然也被覆盖。

外切面与圆柱体上下底面分别交于两点（如图 2-3），由对称性仅以上顶点为例分析。连接导弹与上顶点，该直线记为 L 。若 L 与烟雾半球无交点，则由光的直线传播，该侧未完全遮蔽；若 L 与半球相切，则此时为该侧恰被遮蔽之临界状态；若 L 与半球相交，则该侧已被完全遮蔽，但该时刻并非最早完全遮蔽时间。

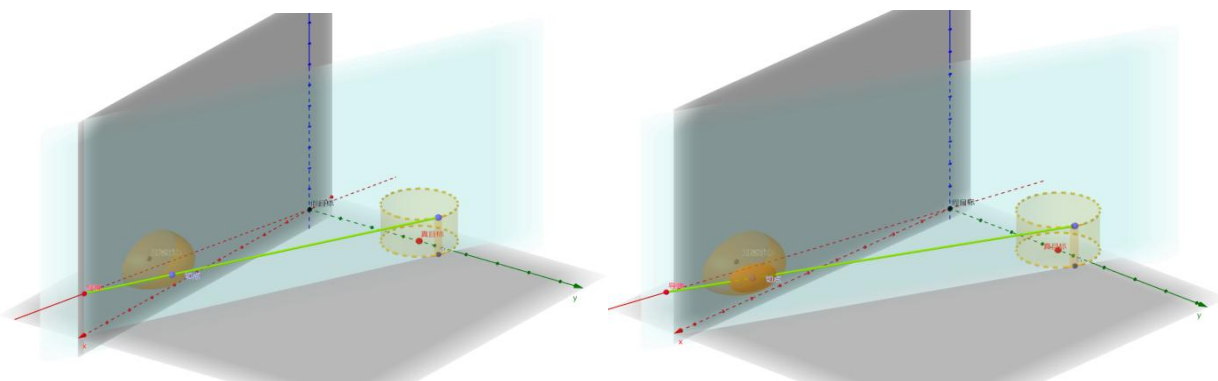


图 2-3 导弹-上顶点连线与球体相交情况示意图

(4) 上下半侧有效阴影区域的遮蔽分析

由上述分析，对任一半侧有效阴影区域，直线与半球相切时，对应其最长遮蔽时间。通过求解方程组可得两个对应时间 t ，并应进行判别：

若取 t 值较小，则对应半侧恰好开始遮蔽，而另一侧尚未达到遮蔽条件，此时整体未完全遮蔽；

若取 t 值较大，则一侧已处于遮蔽状态，另一侧因已过其临界时刻而处于相交状态，此时目标被完全遮蔽。

若两 t 值相等，则表明两侧同时达到临界遮蔽状态。

基于上述模型，代入给定参数，可通过解析方法求出临界时间 t 。再代入表达式 2-1，所得即为系统对目标的完全遮蔽时间。（如图 2-4）

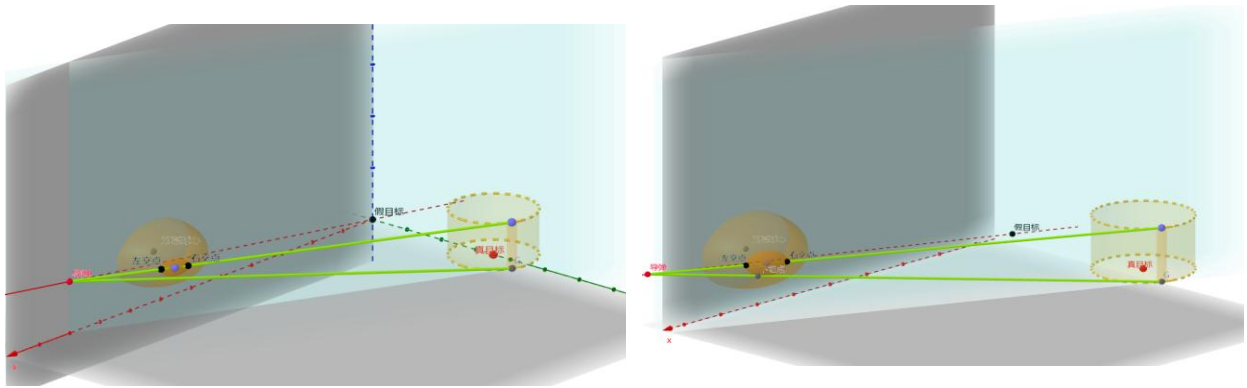


图 2-4 在较短 t 值与较长 t 值情况下导弹-顶点连线与球体相交情况示意图

5.2.5 几何法最终求解过程

设自烟雾产生起至任意时间节点的时间为 t 。

由于平面与目标圆柱体相切，故该平面与整个圆柱体 $x^2 + (y - 200)^2 = 49$ 相切于一条母线。过切点 $(18477.6 - 298.51t, 0, 1847.76 - 29.85t)$ 作垂直于圆柱母线的平面，如图 2-5，由勾股定理可得如下几何关系：

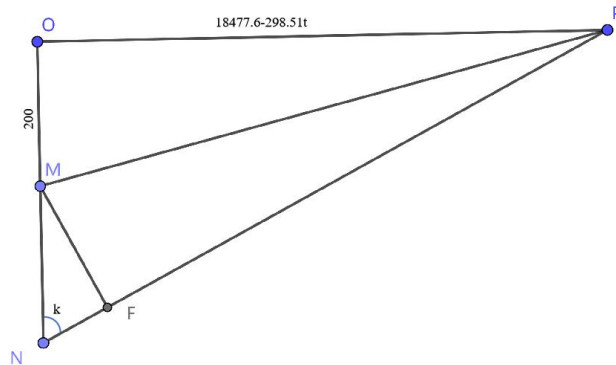


图 2-5 几何法求解有效遮蔽时长示意图

$$49[(200 + k)^2 + (18477.6 - 298.51t)^2] = k^2(18477.6 - 298.51t)^2 \quad (2-2)$$

由此可知，切线段与圆柱上下底面的交点分别为

$$\left(\frac{49}{k}, 200 + \frac{7\sqrt{k^2 + 49}}{k}, 10 \right)$$

$$\left(\frac{49}{k}, 200 + \frac{7\sqrt{k^2 + 49}}{k}, 0 \right)$$

分别求解这两个点与点 $(18477.6 - 298.51t, 0, 1847.76 - 29.85t)$ 的连线直线方程。随后，分别求解这两条直线与以烟雾中心为球心、半径为 5 米的球面之间的距离等于球半径的方程。以上两个表达式均为关于时间 t 的方程，所得取较大者，即 $t = 2.938$ s。

再次利用勾股定理，有： $S_{\text{总}} = 1303.37$ ， $v = 299.72\text{m/s}$ 。

由此解出导弹穿越烟雾的总时间：

$$t_{\text{总}} = \frac{S_{\text{总}}}{v} = 4.3486 \text{ s} \quad (2-3)$$

进而得最终结果，有效遮蔽时间为：

$$\tau = t_{\text{总}} - t_1 = 1.4106 \text{ s} \quad (2-4)$$

至此，我们已通过几何方法，得到了明确的有效遮蔽时长结果。为验证其正确性，接下来采用离散时间遍历算法对上述值进行高精度独立计算，并简要展示关键模型的建立与求解，以便明晰算法奏效的支撑逻辑。

5.2.6 导弹遮蔽模型

在离散时间步长 Δt 内，将导弹轨迹近似为线段：

$$L(s) = M(t - \Delta t) + s \cdot (M(t) - M(t - \Delta t)), s \in [0,1] \quad (2-5)$$

其中 $M(t - \Delta t)$, $M(t)$ 分别为导弹在 $(t - \Delta t)$, t 时刻的位置。

烟幕在三维空间中用球体方程表示为：

$$\|L(s) - C(t)\|^2 \leq r^2 \quad (2-6)$$

$C(t)$ 为 t 时刻烟幕中心， r 为烟幕半径。轨迹点落入球体内部即视为被遮蔽。

定义轨迹被遮蔽的比例函数为：

$$\alpha(t) = \text{segment_sphere_intersection}(M(t - \Delta t), M(t), C(t), r) \in [0,1] \quad (2-7)$$

完全遮蔽的判定函数如下：

$$\chi(t) = \begin{cases} 1, & z_{\text{smoke}} \geq 2 \text{ and } \alpha(t) = 1 \\ 0, & \text{otherwise} \end{cases} \quad (2-8)$$

最终，总有效遮蔽时长为：

$$\tau = \int_{t_{\text{start}}}^{t_{\text{end}}} \chi(t) dt \quad (2-9)$$

5.2.7.验证实验设置

【导弹信息】

导弹初始位置：(20000, 0, 2000)

导弹飞行方向向量：(-0.9950, 0, -0.0995)

【时间范围】

起爆时刻：5.10 s

有效时间窗口：[5.10 s, 25,10 s]（共计 2002 个时间步）

5.2.8 验证结果

经遍历算法逐时间步计算，真目标被有效遮蔽的总时长为：**1.3900 s**，与几何方法所得值高度一致，有力验证了几何模型的准确性。

5.3 问题 2 模型建立与求解

5.3.1 模型建立

基于对烟幕干扰弹投放物理特性与战术需求的深入分析，我们提出了时空协同优化模型。该模型的核心是对无人机的飞行轨迹、速度等参数，以及干扰弹的投放和起爆时序进行协同优化。其最终目标，是使生成的烟幕云团能够在导弹飞行路径与被保护目标之间，形成持续时间最长的有效遮蔽屏障。

(1) 目标函数

我们使用如下适应性函数，完成对目标最长有效遮蔽时间的准确求解：

$$\max_t f(\theta, v, t_1, t_2) = \int_{t_{start}}^{t_{end}} S(t)dt + B(\theta, v, t_1, t_2) \quad (3-1)$$

该目标函数中的**遮蔽函数** S 为二值函数，可**决定遮蔽时间积分的有效性**，满足：

$$S(t) = \begin{cases} 1, & \text{若目标被半径为 } r \text{ 的烟雾完全遮挡} \\ 0, & \text{otherwise} \end{cases} \quad (3-2)$$

若导弹—目标视线完全落在烟幕半径 r 内，则函数值为1，否则为0。

边界奖励项 B 则旨在**引导参数逼近边界解**，如特定速度值或极短延时。该项具体满足如下函数关系：

$$B(\theta, v, t_1, t_2) = \begin{cases} 0.1, & |v - 70| < 1 \text{ or } |v - 140| < 1 \\ 0, & \text{otherwise} \end{cases} \quad (3-3)$$

此机制不影响遮蔽判定，但可**增强边界区域的奖励激励**，进而使搜索算法在边界附近的探索概率显著提升。

(2) 约束条件

我们构建的模型，其约束条件主要包括以下几方面：

1. 无人机投放速度约束

$$70 \leq v \leq 140 \quad (3-4)$$

该约束保证了无人机须在区间内保持某一恒定速度直线飞行。

2. 起爆高度约束

$$z_{start} = z_0 - \frac{1}{2}gt_2^2 \geq 5 \quad (3-5)$$

其中 z_0 为无人机初始高度， z_{start} 为起爆点高度。该约束保证了起爆点高度维持在安全阈值以上。

3. 时间顺序约束

$$t_{start} = t_1 + t_2 < t_{end} = \min(t_{start} + t_{smoke_valid}, t_{missile_arrival}) \quad (3-6)$$

其中 t_{start} 为起爆时刻， t_{smoke_valid} 为烟幕有效持续时间， $t_{missile_arrival}$ 为导弹到达目标时

刻, t_{end} 为实际有效遮蔽结束时刻。该约束保证了在烟幕有效作用时段结束或导弹到达目标前, 烟雾干扰弹能起爆。

4. 烟幕动态高度约束

$$z_{\text{smoke}}(t) = z_{\text{start}} - v_{\text{sink}} \cdot (t - t_{\text{start}}) \geq 2, \forall t \in [t_{\text{start}}, t_{\text{end}}] \quad (3-7)$$

z_{smoke} 为时间 t 时烟幕中心高度, v_{sink} 为烟幕下降速度。该约束保证了导弹在烟幕区域内被完全遮蔽。

5.3.2 模型求解

我们采用改进粒子群优化算法 (PSO) 求解上述模型。该算法在标准粒子群优化基础上, 引入了自适应权重调整机制和边界奖励策略, 尤其适合解决非线性、多约束优化问题。其基本思想在于通过群体中个体之间的协作和信息共享来寻找最优解[3]。在该问情境下, 每个粒子通过位置向量 $(\theta, v, t_{\text{drop}}, t_{\text{burst}})$ 编码一组完整的烟幕干扰弹投放策略, 分别表示无人机的飞行方向角、飞行速度、投放延迟时间及起爆延迟时间。核心求解见图 3-1:

具体步骤为:

Step1 初始化粒子群及参数

设定粒子群规模为 50, 最大迭代次数为 100, 认知系数 c_1 与社会系数 c_2 均取值为 2, 始末惯性权重 $w_{\text{start}}, w_{\text{end}}$ 分别为 0.9 和 0.4。根据变量边界 **bounds**, 在各维度范围内, 随机生成粒子初始位置 **positions**, 并初始化粒子速度为边界范围的 $\pm 10\%$ 。随后计算各粒子的初始适应度值, 并更新个体最优位置与全局最优位置。

Step2 计算粒子适应度

每次迭代中, 计算当前粒子位置的适应度值, 并记录当前群体适应度历史 **pop_history**。

Step3 更新个体与全局最优解

对每个粒子, 若其当前适应度优于个体历史最优适应度 $p_{\text{best_fitness}}$, 则更新该粒子的个体最优位置与适应度; 若当前适应度优于全局最优适应度 $g_{\text{best_fitness}}$, 则更新全局最优位置与适应度。

Step4 更新粒子速度与位置

根据粒子群优化算法速度更新公式:

$$v_{\text{new}} = w \cdot v_{\text{old}} + c_1 \cdot r_1 \cdot (p_{\text{best}} - x) + c_2 \cdot r_2 \cdot (g_{\text{best}} - x) \quad (3-8)$$

基于动态惯性权重 w , 学习因子 c_1, c_2 , 以及[0,1]范围内的随机数 r_1, r_2 , 我们可计算出各粒子的新速度, 并对速度进行约束处理, 确保其绝对值不超过边界范围的 20%。再依据新速度更新粒子位置:

$$x_{\text{new}} = x_{\text{old}} + v_{\text{new}} \quad (3-9)$$

并通过边界约束确保位置处于可行解范围内。

经过 100 次迭代优化后, 算法已收敛至最优解。最优参数组合为: 无人机飞行方向角 (角度制表示) $\theta = 8.29^\circ$, 飞行速度 $v = 70.0043 \text{ m/s}$, 投放延迟时间 $t_{\text{drop}} = 0.0022 \text{ s}$,

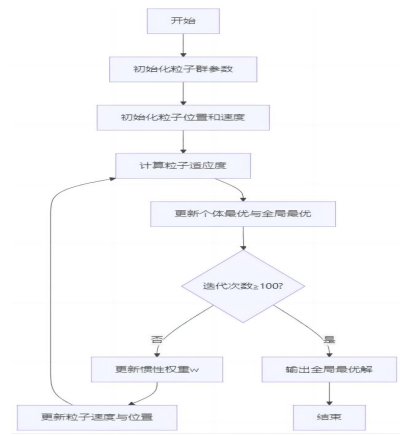


图 3-1 改进粒子群优化算法的流程图

起爆延迟时间 $t_{burst} = 0.9999$ s。相应投放点坐标为 (17800.1523, 0.2220, 1800.0000)，起爆点坐标为 (17869.4202, 10.1147, 1795.0957)。

高精度验证计算表明，该最优策略可实现的有效遮蔽时间为：

$$\tau = 4.80 \text{ s}$$

优化结果符合理论预期，较低的飞行速度（接近下限 70 m/s）与极短的投放延迟（0.0022 s）有助于在理想时空位置形成有效烟幕遮蔽。边界奖励策略成功引导算法收敛至该边界解，证明了策略设计的有效性。这一结果为指挥决策提供了科学的定量依据，实现了烟幕干扰弹投放策略的优化设计。

为验证该优化结果，我们做了进一步实验，并得到了如下对比图（图 3-2）。

该图对比了 PSO、GA、DE、SA 和 GWO 等优化算法在不同迭代次数下的收敛性能，以迭代次数为横轴，适应值为纵轴。结果显示，多数算法再收敛速度和精度均上表现不佳，尤其 SA 与 GWO 初期适应值低，收敛缓慢。而改进 PSO 作为优选算法，能以较少迭代实现高精度与稳定收敛，适用于高效优化场景。

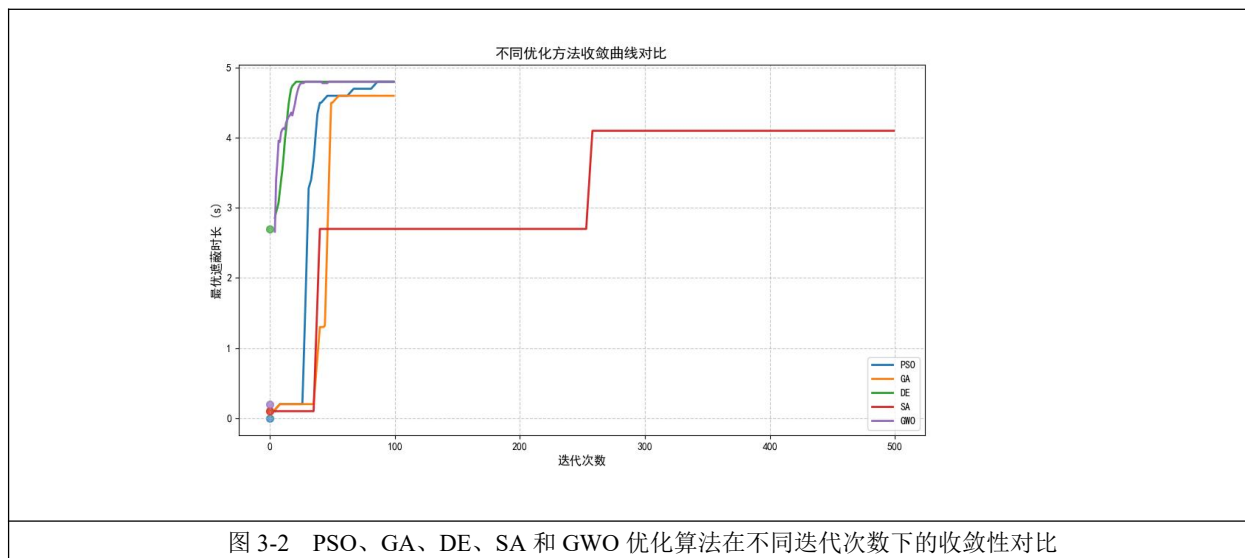


图 3-2 PSO、GA、DE、SA 和 GWO 优化算法在不同迭代次数下的收敛性对比

5.4 问题 3 模型建立与求解

5.4.1 对问题的简化分析

(1) 无人机飞行方向的确定

在第一问中，无人机原设定朝向原点等高度飞行，与导弹在 x 轴方向速度分量方向一致。然而，若按此方向投放三个烟雾弹，由于烟雾弹竖直速度仅为 3 m/s，而导弹有 29.85 m/s，第一个烟雾弹形成遮蔽后，后续烟雾弹无法在导弹到达同一 x 轴坐标时，及时到达相应高度以实现遮蔽。因此，将策略调整为使无人机在 x 轴的速度分量与导弹完全相反，从而使三个烟雾弹均有可能对导弹形成有效遮蔽。

(2) 引爆时间的忽略

烟雾弹投放后经历平抛运动才引爆下落，但因无人机与导弹在 x 轴方向速度分量相反，该过程会导致烟雾弹产生较大水平位移，难以在导弹到达时形成有效遮蔽。因此，我们将模型简化为烟雾弹投放后立即引爆，并仅以 3 m/s 匀速下落，以更好地匹配导弹

轨迹。

（3）首枚烟雾弹投放时刻的确定

导弹与无人机轨迹交汇于点（18000，0，1800），故三枚烟雾弹的投放需限制在该点周围 200m×20m 的区域内。若首枚烟雾弹投放过晚，将致后续烟雾弹超出有效范围，难以实现对导弹的长时间遮蔽。因此，我们采取的策略为在无人机起飞后，立即投放烟雾弹，尽可能延长三枚烟雾弹的整体遮蔽时间。（如图 4-1）

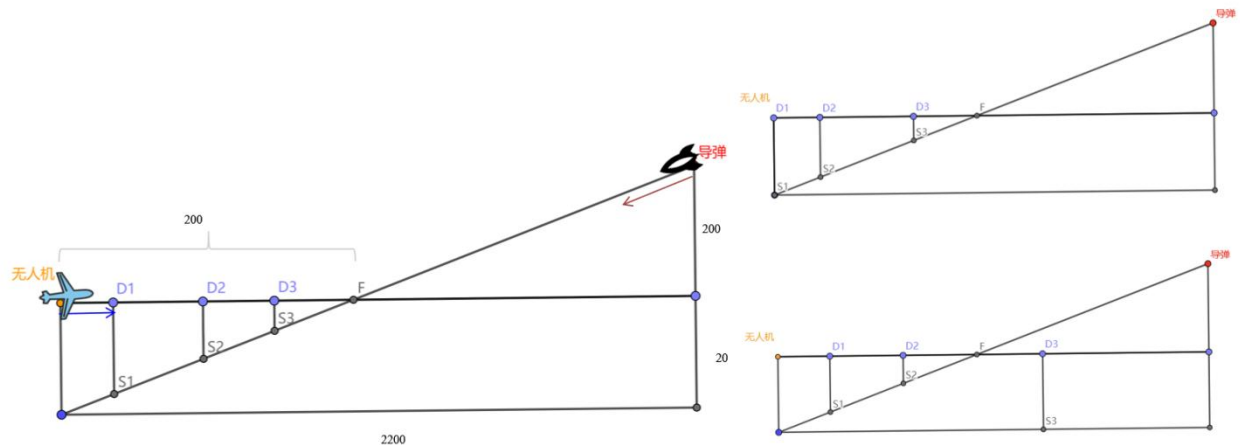


图 4-1 无人机 FY1 投放 3 枚烟幕干扰弹示意图

（4）完全遮蔽最优情形分析

基于第一问的相对运动模型，在以烟雾弹为参考系中，导弹速度可合成为一个朝 z 轴正方向、大小为 3 m/s 的分量。不同投放策略下，导弹运动轨迹与烟雾区域存在七种相对位置关系。当由导弹位置引出的两切平面所构成的有效阴影区域，能等比例投影至烟雾遮蔽范围内，且导弹轨迹恰好穿过烟雾球心时，完全遮蔽时间取得最大值。因此，我们尽可能使导弹轨迹穿过烟雾球心，或至少最小化其穿过时与球心的距离。（如图 4-2）

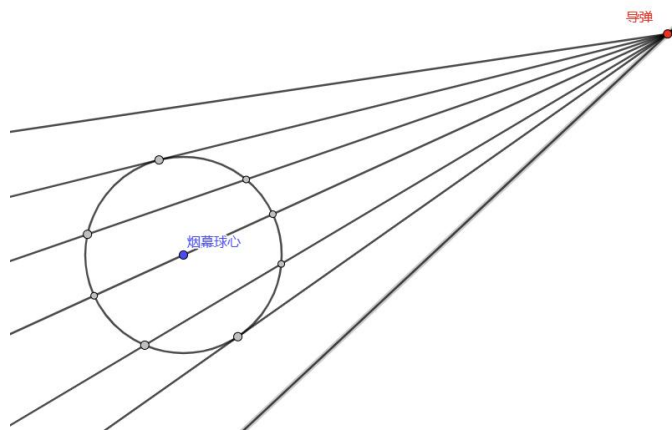


图 4-2 导弹运动轨迹与烟雾区域七种相对位置关系示意图

5.4.2 多枚烟雾弹遮蔽效果评估

若三个烟雾弹均位于导弹轨迹附近，距离越近的烟雾弹所形成的有效遮蔽范围越大。故需引入权重机制，优先确保第三枚烟雾弹的球心被导弹轨迹穿过，其次为第二枚，最后为距离最远的第一枚。（如图 4-3）

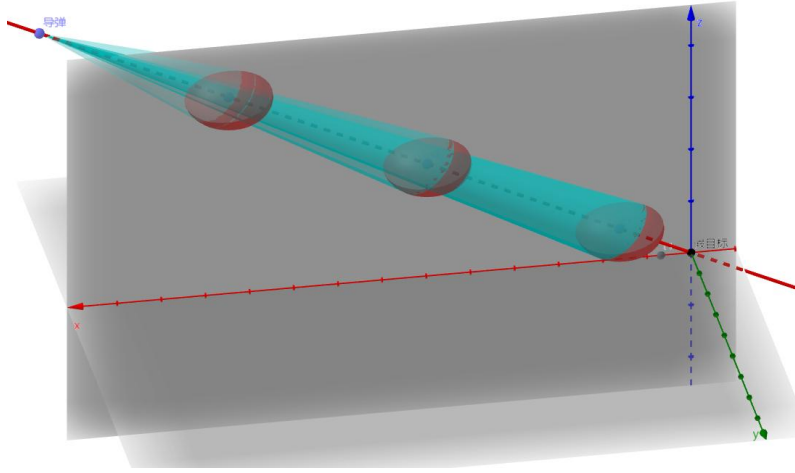


图 4-3 烟雾干扰弹距离导弹所在点不同远近的遮蔽效果示意图

5.4.3 烟雾弹与导弹相对位置的建模分析

根据前述分析，第一个烟雾弹在投放后立即引爆。设经过时间 Δt_1 后投放第二个烟雾弹，再经过 Δt_2 后投放第三个烟雾弹。第三个烟雾弹的下落时间记为 t_d ，无人机飞行速度设为 v 。

初始时刻，导弹的运动时间 $T_1 = \Delta t_1 + \Delta t_2 + t_d$ ，坐标为

$$(20000 - 298.5T_1, 0, 2000 - 29.85T_1) \quad (4-1)$$

此时我们只考虑最后投放的第三个烟雾弹所产生的烟雾坐标，为

$$(17800 + v(\Delta t_1 + \Delta t_2), 0, 1800 - 3t_d) \quad (4-2)$$

第二个时刻，导弹在水平方向的分速度经过了 $v \cdot \Delta t_2$ 这一段路程，运动时间

$$T_2 = \Delta t_1 + \Delta t_2 + t_d + \frac{v \cdot \Delta t_2}{298.5}, \quad (4-3)$$

可以写出导弹坐标为

$$(20000 - 298.5T_2, 0, 2000 - 29.85T_2) \quad (4-4)$$

此时我们只考虑最后投放的第三个烟雾弹所产生的烟雾坐标，为

$$(17800 + v \cdot \Delta t_1, 0, 1800 - 3(\Delta t_2 + t_d + \frac{v \cdot \Delta t_2}{298.5})) \quad (4-5)$$

第三个时刻，导弹在水平方向的分速度又经过了 $v \cdot \Delta t_1$ 这一段路程，运动时间 $T_3 = \Delta t_1 + \Delta t_2 + t_d + \frac{v(\Delta t_2 + \Delta t_1)}{298.5}$ ，可以写出导弹坐标为

$$(20000 - 298.5T_3, 0, 2000 - 29.85T_3) \quad (4-6)$$

此时我们只考虑最后投放的第三个烟雾弹所产生的烟雾坐标，为

$$(17800, 0, 1800 - 3(\Delta t_1 + \Delta t_2 + t_d + \frac{v(\Delta t_1 + \Delta t_2)}{298.5})) \quad (4-7)$$

5.4.4 目标函数的建立

根据前述分析，当导弹轨迹同时穿过三个烟雾弹的球心时，遮蔽时间达到最大值。然而，实际情况下难以实现完全穿过，因此我们引入以下距离度量：第一个时刻，导弹与烟雾球心的距离记为 L_1 ，第二个时刻相应记为 L_2 ，第三个相应记为 L_3 。

理想情况下,三个距离都为 0。实际中则需使其最小化。考虑到不同次序释放的烟雾弹对遮蔽效果的影响不同（第三枚效果最佳，第一枚最差），引入权重系数 α, β, γ 以区别其重要性，建立如下目标函数：

$$\min \alpha \cdot L_1 + \beta \cdot L_2 + \gamma \cdot L_3$$

同时，注意到约束条件有：

$$s.t. \begin{cases} \Delta_{t1} > 1s, \Delta_{t2} > 1s \\ 70 \leq v \leq 140 \\ A_z \geq 0, B_z \geq 0, C_z \geq 0 \end{cases} \quad (4-8)$$

其中需确保无人机速度维持在 70 ~ 140 m/s 之间，投弹间隔不小于 1 s，且无人机释放点与导弹目标点的 z 坐标均非负。

通过数值实验与遮蔽时间比较，确定影响系数为：

$$\alpha = 150, \beta = 37.5, \gamma = 1$$

在满足上述约束的条件下进行优化，得到三个烟雾弹爆炸点坐标分别为：

(17800.000, 0, 1777.7791), (17913.397, 0, 1791.9517), (17799.274, 0, 1779.9274)

最终求得，最大遮蔽时间

$$\tau_{\max} = 6.51 \text{ s}$$

5.5 问题 4 模型建立与求解

5.5.1 模型建立

(1) 目标函数

针对该多无人机资源调度问题，我们引入价值系数 v_{ij} 以协同衡量理论遮蔽潜力与实际执行成本。

首先，通过求解单无人机—单导弹子问题，暂不考虑其他约束，得到无人机 i 对导弹 j 所能提供的最大遮蔽时长：

$$\tau_{ij}^{\max} = \max_{x_i} \int_0^T B_j(t; x_i) dt \quad (5-1)$$

随后，计算机动代价。具体来说，计算无人机 i 从其初始位置 u_i^0 移动至可实现 $\tau_{ij}^{\max}$ 的理想投放点所需距离：

$$d_{ij} = \|x_i^* - u_i^0\| \quad (5-2)$$

在此基础上，价值系数 v_{ij} 被定义为最大遮蔽时长与移动距离惩罚的比值，即：

$$v_{ij} = \frac{T_{ij}^{\max}}{\beta + d_{ij}} = \frac{\max_{x_i} \int_0^T B_j(t; x_i) dt}{\beta + \|x_i^* - u_i^0\|} \quad (5-3)$$

其中， β 为距离惩罚系数。

进一步地，构建适应度函数，以最大化总价值并惩罚约束违反：

$$\text{fitness}(X) = - \left(\sum_{i=1}^{N_{\text{UAV}}} \sum_{j=1}^{N_m} z_{ij}(X) \cdot v_{ij} \right) + \alpha \cdot \sum_c \max(0, \text{violation}_c(X)) \quad (5-4)$$

其中解向量 X 编码了所有无人机的部署位置，而函数 $z_{ij}(X)$ 则根据位置解 X 所确定的遮蔽效能动态决策任务分配关系。最后，通过鲸鱼优化算法（WOA）最小化该适应度函数，可同步求解出最优的无人机部署与任务指派策略。

(2) 约束条件

$$\begin{cases} \|x_i - u_i^0\| \leq \text{MAX_DIST}, \forall i \\ \|x_i - o_k\| \geq d_{\text{safe}}, \forall i, k \\ \|x_i - x_l\| \geq d_{\text{safe}}, \forall i \neq l \end{cases} \quad (5-5)$$

如上三式表示无人机投放点的选择需满足以下约束：

- 1.移动范围约束：**各无人机的投放点必须位于其初始位置的最大可移动距离范围内；
- 2.避障约束：**各投放点需与所有静态障碍物保持最小安全距离；
- 3.防碰撞约束：**任意两个无人机投放点之间需保持最小间隔距离，避免碰撞；
- 4.空间可行性约束：**每个投放点须为三维空间中的可行位置。

5.5.2 模型求解

鲸鱼优化算法（WOA）具备卓越的全局探索与局部开发平衡能力、较少的调节参数以及强大的避免局部最优能力，因而我们将其作为本模型的优选求解框架。算法模仿座头鲸泡泡网捕食的智能搜索机制，尤其适合处理本问题中决策变量，即无人机飞行方向、速度、投放和起爆延时等变量，与目标遮蔽时间的复杂非线性关系。此外，该算法结构简洁，易于实现和调整。如下呈现该算法的核心流程（见流程图 5-1）：

Step1 初始化函数及参数

设定种群个体数量为 60，最大迭代次数为 400，初始化参数 $a = 2$ ，并在无人机可达范围 $[X_{\min}, X_{\max}]$ 内随机生成初始解 X_i ，每个解均表示无人机投放点坐标。

Step2 计算个体适应度

对每个个体 X_i ，计算适应度函数：

$$f(X_i) = - \sum_{u=1}^5 \sum_{m=1}^3 \text{Coverage}(u, m, X_i)$$

其中，

$$\text{Coverage}(u, m, X_i) = w_{um} \cdot \exp\left(-\frac{\|x_i - p_{-1}\|}{5000}\right)$$

表示当前无人机位置对第 u 个导弹发射点、第 m 枚导弹的遮蔽效果。

Step3 计算个体与全局最优解

将当前各个体的适应度值与自身历史最优值进行比较，更新个体最优解；再在所有个体最优解中选取最佳解，更新全局最优解 BestWhale。

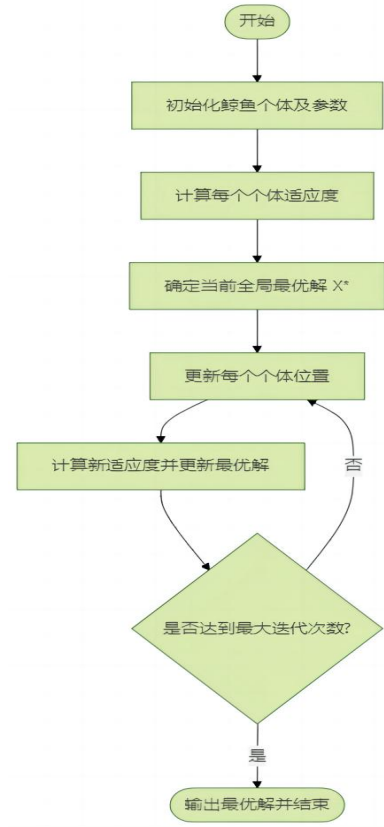


图 5-1 鲸鱼优化算法 (WOA) 流程图

Step4 更新种群位置

根据收缩包围机制与搜索机制更新解：

$$X_i^{t+1} = \begin{cases} X^* - A \cdot |C \cdot X^* - X_i^t|, & |A| < 1 \quad (\text{收缩包围}) \\ X_{\text{rand}} - A \cdot |C \cdot X_{\text{rand}} - X_i^t|, & |A| \geq 1 \quad (\text{搜索机制}) \end{cases}$$

其中 $A = 2ar_1 - a$, $C = 2r_2$, r_1, r_2 为随机数。

Step5 判断是否达到最大迭代次数

若未达到，则继续 Step2；若达到，则输出全局最优解 X^* 作为最终无人机投放点方案，终止算法。

WOA 算法的搜索机制与本模型的时空协同优化特性高度契合。其“包围猎物”阶段实现了对无人机飞行速度、方向角参数的精细调优，而“气泡网攻击”的螺旋更新行为，则有效协调了投放延时与起爆延时两个关键时序变量的组合优化。同时，算法内置的随机搜索算子显著增强了全局探索，也即逃离局部极值的能力，这对于应对遮蔽时间函数非线性、多峰特性的挑战一样尤为关键。通过将无人机的机动范围约束、避障及防碰撞约束以罚函数形式融入适应度评价，WOA 的搜索过程被系统地约束于可行域内。

经过运算，最终得到任意 3 架无人机各投放 1 枚烟幕干扰弹实施对 M1 干扰的投放策略见汇总表 5-1。

表 5-1 任意 3 架无人机各投放 1 枚烟幕干扰弹实施对 M1 干扰的投放策略汇总表

组合	总覆盖 时长(s)	第一架飞机		第二架飞机		第三架飞机	
		坐标	时长(s)	坐标	时长(s)	坐标	时长(s)
FY1,FY2,FY3	5.06	(19991, -57, 2092)	4.5	(10650, 1777, 7741)	0.33	(11525, 5179, 8240)	0.23
FY1,FY2,FY4	8.72	(20000, 0, 2000)	4.6	(19994, 0, 1997)	3.08	(14213, 510, 306)	1.04
FY1,FY2,FY5	8.17	(20000, 0, 2000)	4.6	(20039, 260, 2052)	2.92	(15038, 578, 350)	0.66
FY1,FY3,FY4	4.68	(19994, 593, 2111)	4.08	(8847, 5103, 5949)	0.22	(9120, 1868, 898)	0.38
FY1,FY3,FY5	5.42	(20018, 162, 1928)	4.44	(11017, 5574, 1404)	0.35	(15987, -524, 5623)	0.63
FY1,FY4,FY5	4.71	(16784, 3806, 6807)	1.15	(19998, 4, 2021)	3.48	(3843, -595, 3819)	0.07
FY2,FY3,FY4	4	(19994, -109, 2130)	2.98	(8490, 11398, -1591)	0.07	(15997, 4643, -228)	0.95
FY2,FY3,FY5	4.95	(20000, 113, 1956)	3.01	(19665, 1473, 2647)	1.45	(13671, 1822, 829)	0.49
FY2,FY4,FY5	3.98	(20002, 73, 2075)	3.02	(13087, -1158, 5105)	0.76	(13500, 4487, 9312)	0.21
FY3,FY4,FY5	4.13	(25142, -441, -463)	0.48	(19980, -13, 2069)	3.45	(8831, -297, 1053)	0.2

综上，由该算法得到的解方案既实现了遮蔽时间的最大化，又完全满足了战术可行性要求。WOA 在本模型求解中展现出了卓越的求解精度和鲁棒性，确为解决该复杂时空协同决策问题的有效工具。

5.6 问题 5 模型建立与求解

5.6.1 模型建立

(1) 价值系数矩阵

在第 4 问中，价值系数 v_{ij} 作为评估多架无人机 i 遮蔽单—导弹 j 潜在效能的权重指标，成功指导了单导弹场景下的无人机调度。在第五问中，我们扩展这一概念，通过构建价值系数矩阵 $\mathbf{V} = [v_{ij}]_{N_u \times N_m}$ 来实现多导弹场景下的无人机群智能划分，从而将复杂的多对多协同问题简化为多个可并行求解的多无人机-单导弹问题。

价值系数 v_{ij} 采用结构化分解形式计算，综合考虑遮蔽收益、部署成本和冗余惩罚：

$$v_{ij} = w_j \cdot B_{ij} - \lambda_{cost} \cdot \overline{Cost}_{ij} - \lambda_{redund} \cdot \overline{Redund}_{ij} \quad (6-1)$$

各项均经过归一化处理至 $[0,1]$ 区间， λ 系列为平衡权重超参数。

其中， B_{ij} 表示单无人机单导弹问题下，无人机 i 对导弹 j 的最大预期遮蔽时长：

$$B_{ij} = \max_{\theta_1 \in \theta_i} \int_0^{T_j} \delta_{ij}(t; \theta_i) dt \quad (6-2)$$

部署成本 $\overline{Cost}_{ij}$ 指无人机 i 为执行遮蔽导弹 j 的任务所需付出的机动代价，通常用从其初始位置 u_i^0 机动至最优投放点 x_i^* 的欧式距离表示

$$Cost_{ij} = \|x_i^* - u_i^0\| \quad (6-3)$$

其中：

$$x_i^* = \arg \max_{x_i} \int_0^T B_j(t; x_i) dt \quad (6-4)$$

即通过求解单机单目标问题得到的最优投放点。

(2) 价值驱动的协同多因子任务分配算法

基于价值系数矩阵 V ，我们构建带容量约束的分配模型：

$$\max_{z_{ij}} \sum_{i=1}^{N_u} \sum_{j=1}^{N_m} v_{ij} z_{ij}, \text{ s. t. } \sum_{i=1}^{N_u} z_{ij} \geq 1, \sum_{j=1}^{N_m} z_{ij} \leq q_i, z_{ij} \in \{0,1\} \quad (6-5)$$

其中 q_i 表示无人机 i 的任务容量（即可投放的烟幕弹数量）。该模型保证每枚导弹至少被一架无人机覆盖，且每架无人机的任务数不超过其容量限制。

求解此分配模型得到最优分配方案 z_{ij}^* 后，以导弹 j 为单位划分专属无人机子集：

$$U_j = \{i | z_{ij}^* = 1, i = 1, 2, \dots, N_u\} \quad (6-6)$$

至此，原始的多对多协同遮蔽问题成功分解为 N_m 个独立的多无人机—单导弹问题。每个子问题可在第四问建立的模型上扩展决策变量与约束条件与 WOA 优化方法并行求解，针对导弹 j 优化其专属无人机子集 U_j 的投放策略，最大化遮蔽效果。

5.6.2 模型求解

基于多维度数据矩阵及空间坐标参数分析，价值系数矩阵分布呈现显著几何约束效应与效能权衡关系：无人机 FY1 因与导弹 M2 的空间邻近性（部署成本 1374.77 米、遮蔽时长 7.686 秒）产生正向协同效应，展现 0.915 的最优价值系数；FY5 受与目标集群几何关系影响，遮蔽效能过低致所有价值系数为负，作战效能存在根本局限；FY2 与 FY4 虽有中等遮蔽能力，但部署成本均超 7000 米，受距离衰减效应影响系统净收益为负；冗余惩罚矩阵同行数据完全一致的均匀分布，表明当前基于平均距离的简化计算模型未充分捕捉无人机编队间时空交互特性。该价值系数矩阵从数学上量化了空间几何约束对作战效能的影响规律，为协同任务分配提供多目标优化决策依据，且空间位置参数对系统价值的预测能力强于绝对性能参数。价值系数矩阵热力图见图 6-1。

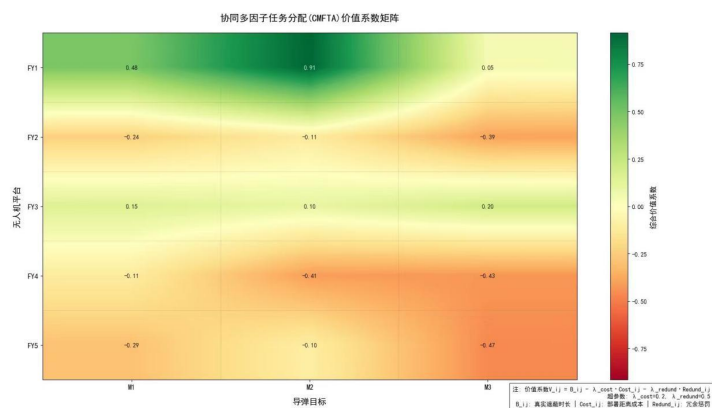


图 6-1 价值系数矩阵热力图

表 6-1 5 架无人机每架至多投放 3 枚烟幕干扰弹首次投放策略汇总表

无人机编号	投放点		
	x	y	z
FY1	19184.1	2000.0	1919.6
FY2	19178.0	3000.0	2640.0
FY3	966.9	1405.1	899.4
FY4	1343.4	1909.9	966.9
FY5	1047.1	8751.6	978.1
最大遮蔽时长	14.7325 s		

本算法通过多维度空间分析，有效揭示了多无人机—多导弹的综合情境下，无人机协同作战的效能规律。其核心价值在于证实了空间位置参数对系统作战效能的预测能力显著优于绝对性能参数，凸显了协同任务中时空配置的决定性作用。此外，模型还量化了不同空间关系下的协同效应：部分单元因位置优势形成高效协同，部分单元因布局问题丧失效能，另有单元因距离因素收益受限。价值系数矩阵从数学层面明确了空间几何约束与作战效能的内在关联，为此高难度协同任务分配提供了可靠的多目标优化依据。

六、模型评价与推广

6.1 模型优点

6.1.1 多智能体协同优化能力

在多无人机、多干扰弹、多导弹的复杂场景下，模型可有效协调各单元的时空行为，实现遮蔽时间最大化，体现了较强的系统协调与优化能力。

6.1.2 良好的扩展性与适应性

模型可灵活适应不同数量的无人机、干扰弹和导弹，具备较强的泛化能力，便于在不同想定背景下推广应用。

6.2 模型缺点

6.2.1 环境因素假设过于理想化

模型中假设烟幕云团为理想球形、匀速下沉，未考虑风速、湿度、大气湍流等环境因素的影响，实际应用中可能存在偏差。

6.2.2 烟幕扩散模型过于简化

采用 10m 范围内的阈值模型判断有效遮蔽，未考虑浓度梯度、扩散非均匀性等因素，可能导致遮蔽时间估计偏乐观。

6.3 模型推广与应用

6.3.1 多类型干扰弹集成应用

本模型可扩展至红外干扰弹、箔条弹等其他类型的干扰弹，只需调整其扩散模型和干扰机制，具备较强的通用性。

6.3.2 复杂环境下的适应性优化

可引入气象数据（风速、温度梯度）修正烟幕扩散模型，提升模型在复杂环境下的鲁棒性和适用性。

6.3.3 动态目标与多导弹协同干扰

可进一步研究对移动目标的遮蔽策略，以及多枚导弹同时来袭时的协同干扰方案，提升模型的实战应用价值。

6.3.4 智能决策系统集成

本模型可作为无人集群智能决策系统的一部分，结合强化学习、多智能体协同算法，实现更高层次的自主决策与动态调整。

6.3.5 民用领域应用拓展

类似模型也可用于消防烟雾投放、农业喷洒、环境监测等民用领域的时空资源调度问题，具备广泛的跨领域应用前景。[4]

七、参考文献

- [1] 刘娜,毛晓菊.基于分离轴定理的碰撞检测算法[J].数字技术与应用,2012,(08):102.DOI:10.19695/j.cnki.cn12-1369.2012.08.070.
- [2] 司守奎,孙玺菁.数学建模算法与应用[M].北京:国防工业出版社,2022
- [3] 周驰,高海兵,高亮等.粒子群优化算法[J].计算机应用研究,2003(12):7-11..周驰;高海兵;高亮;章万国.
- [4] DeepSeek, DeepSeek-R1-0528, 深度求索 (DeepSeek), 2025-09-05

附录

附录一：问题 1 代码求解

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import matplotlib as mpl

# ----- 1. 参数配置 -----
# 物理常量
G = 9.8 # 重力加速度
EPS = 1e-12 # 数值容差

# 目标位置
FAKE_TARGET = np.array([0.0, 0.0, 0.0]) # 诱饵目标

# 真实目标参数
REAL_TARGET = {
    "center": np.array([0.0, 200.0, 0.0]), # 圆柱底面中心
    "radius": 7.0, # 圆柱半径
    "height": 10.0 # 圆柱高度
}

# 无人机参数
UAV = {
    "start": np.array([17800.0, 0.0, 1800.0]), # 起始位置
    "speed": 120.0, # 飞行速度
    "drop_time": 1.5, # 投弹准备时间
    "detonate_time": 3.6 # 引爆延迟
}

# 烟幕参数
SMOKE = {
    "radius": 10.0, # 有效遮蔽半径
    "sink_speed": 3.0, # 下沉速度
    "duration": 20.0 # 有效持续时间
}

# 导弹参数
MISSILE = {
    "start": np.array([20000.0, 0.0, 2000.0]), # 发射位置
    "speed": 300.0 # 飞行速度
}

DT = 0.01 # 时间步长

# ----- 2. 核心计算函数 -----
def get_drop_point(uav_pos, uav_speed, delay, target):
    """计算烟幕弹投掷点"""
    uav_xy = uav_pos[:2]
    target_xy = target[:2]
    dist = np.linalg.norm(target_xy - uav_xy)

    if dist < EPS:
        direction = np.array([0.0, 0.0])
    else:
        direction = (target_xy - uav_xy) / dist

    travel_dist = uav_speed * delay
    drop_xy = uav_xy + direction * travel_dist

    return np.array([drop_xy[0], drop_xy[1], uav_pos[2]])

def get_detonation_point(drop_pos, uav_speed, delay, gravity, target):
    """计算烟幕弹引爆点"""
```

```

drop_xy = drop_pos[:2]
target_xy = target[:2]
dist = np.linalg.norm(target_xy - drop_xy)

if dist < EPS:
    direction = np.array([0.0, 0.0])
else:
    direction = (target_xy - drop_xy) / dist

horizontal_dist = uav_speed * delay
det_xy = drop_xy + direction * horizontal_dist

fall_height = 0.5 * gravity * delay ** 2
det_z = drop_pos[2] - fall_height

return np.array([det_xy[0], det_xy[1], det_z])

# ----- 3. 目标采样点生成 -----
def create_target_samples(spec, circle_points=60, height_points=20):
    """生成目标表面和内部采样点"""
    points = []
    center = spec["center"]
    r = spec["radius"]
    h = spec["height"]
    center_xy = center[:2]
    bottom = center[2]
    top = center[2] + h

    # 生成角度序列
    angles = np.linspace(0, 2*np.pi, circle_points, endpoint=False)

    # 底面和顶面圆周点
    for z in [bottom, top]:
        for angle in angles:
            x = center_xy[0] + r * np.cos(angle)
            y = center_xy[1] + r * np.sin(angle)
            points.append([x, y, z])

    # 侧面点
    z_levels = np.linspace(bottom, top, height_points, endpoint=True)
    for z in z_levels:
        for angle in angles:
            x = center_xy[0] + r * np.cos(angle)
            y = center_xy[1] + r * np.sin(angle)
            points.append([x, y, z])

    # 内部网格点
    radii = np.linspace(0, r, 5, endpoint=True)
    z_inner = np.linspace(bottom, top, 10, endpoint=True)
    inner_angles = np.linspace(0, 2*np.pi, 12, endpoint=False)

    for z in z_inner:
        for rad in radii:
            for angle in inner_angles:
                x = center_xy[0] + rad * np.cos(angle)
                y = center_xy[1] + rad * np.sin(angle)
                points.append([x, y, z])

    # 中心轴线点
    axis_points = [
        [center_xy[0], center_xy[1], bottom],
        [center_xy[0], center_xy[1], bottom + h/4],
        [center_xy[0], center_xy[1], bottom + h/2],
        [center_xy[0], center_xy[1], bottom + 3*h/4],
        [center_xy[0], center_xy[1], top]
    ]
    points.extend(axis_points)

    return np.unique(np.array(points), axis=0)

```

```

# ----- 4. 几何判定函数 -----
def check_sphere_intersection(start, end, center, radius):
    """检查线段与球体是否相交"""
    seg_vec = end - start
    center_vec = center - start

    a = np.dot(seg_vec, seg_vec)

    # 处理零长度线段
    if a < EPS:
        return np.linalg.norm(center_vec) <= radius + EPS

    b = -2 * np.dot(seg_vec, center_vec)
    c = np.dot(center_vec, center_vec) - radius ** 2

    discriminant = b ** 2 - 4 * a * c
    if discriminant < -EPS:
        return False

    if discriminant < 0:
        discriminant = 0

    sqrt_d = np.sqrt(discriminant)
    s1 = (-b - sqrt_d) / (2 * a)
    s2 = (-b + sqrt_d) / (2 * a)

    # 检查交点是否在线段范围内
    return (s1 <= 1.0 + EPS) and (s2 >= -EPS)

def is_target_covered(missile_pos, smoke_center, smoke_radius, samples):
    """检查目标是否被完全遮蔽"""
    for point in samples:
        if not check_sphere_intersection(missile_pos, point, smoke_center, smoke_radius):
            return False
    return True

# ----- 5. 可视化函数 -----
# 设置中文字体
mpl.rcParams['font.sans-serif'] = ['SimHei']
mpl.rcParams['axes.unicode_minus'] = False

def check_point(point, name):
    """验证点坐标格式"""
    if not isinstance(point, (list, tuple, np.ndarray)) or len(point) != 3:
        raise ValueError(f"{name} 必须是三维坐标")
    if not all(isinstance(x, (int, float)) for x in point):
        raise ValueError(f"{name} 必须为数值")

def validate_parameters(uav, missile, target):
    """验证参数格式"""
    required_uav = ["start", "speed", "drop_time", "detonate_time"]
    required_missile = ["start"]
    required_target = ["center", "radius", "height"]

    if not all(k in uav for k in required_uav):
        raise ValueError("无人机参数缺少必要字段")
    if not all(k in missile for k in required_missile):
        raise ValueError("导弹参数缺少必要字段")
    if not all(k in target for k in required_target):
        raise ValueError("目标参数缺少必要字段")

    check_point(uav["start"], "无人机起始点")
    check_point(missile["start"], "导弹起始点")
    check_point(target["center"], "目标中心点")

def compute_uav_path(uav_config, drop_point, step=0.1):
    """计算无人机轨迹"""
    check_point(drop_point, "投掷点")
    start = np.array(uav_config["start"])
    speed = uav_config["speed"]

```

```

direction = (drop_point - start)
dist = np.linalg.norm(direction)
time_to_drop = dist / speed if speed != 0 else uav_config["drop_time"]

t = np.arange(0, time_to_drop + step, step)

if dist < EPS:
    unit_dir = np.zeros(3)
else:
    unit_dir = direction / dist

path = start + np.outer(t, unit_dir)
return path

def compute_projectile_path(drop_point, det_point, det_delay, step=0.1):
    """计算抛体轨迹"""
    check_point(drop_point, "投掷点")
    check_point(det_point, "引爆点")
    drop = np.array(drop_point)
    det = np.array(det_point)

    t = np.arange(0, det_delay + step, step)
    v_xy = (det[:2] - drop[:2]) / det_delay if det_delay > 0 else np.zeros(2)
    z = drop[2] - 0.5 * G * t**2
    path = np.zeros((len(t), 3))
    path[:, 0] = drop[0] + v_xy[0] * t
    path[:, 1] = drop[1] + v_xy[1] * t
    path[:, 2] = z
    return path

def compute_smoke_path(det_point, smoke_center, step=0.1):
    """计算烟雾轨迹"""
    if smoke_center is None:
        return None
    check_point(det_point, "引爆点")
    check_point(smoke_center, "烟雾中心")
    det = np.array(det_point)
    center = np.array(smoke_center)

    t = np.arange(0, 1.0 + step, step)
    path = det + (center - det) * (t / 1.0)[:, np.newaxis]
    return path

def plot_uav_trajectory(ax, uav_config, drop_point):
    """绘制无人机轨迹"""
    path = compute_uav_path(uav_config, drop_point)
    ax.plot(path[:, 0], path[:, 1], path[:, 2],
            'b-', label="无人机路径", linewidth=2)

    mid = len(path) // 2
    if mid > 0:
        start = path[mid - 1]
        end = path[mid]
        ax.quiver(start[0], start[1], start[2],
                  end[0] - start[0], end[1] - start[1], end[2] - start[2],
                  color='b', arrow_length_ratio=0.1)

def plot_key_points(ax, drop_point, det_point, smoke_center, det_delay):
    """绘制关键点和轨迹"""
    check_point(drop_point, "投掷点")
    check_point(det_point, "引爆点")
    ax.scatter(*drop_point, c='g', marker='o', s=100, label="投放点")
    ax.scatter(*det_point, c='r', marker='^', s=100, label="引爆点")

    # 抛体轨迹
    projectile = compute_projectile_path(drop_point, det_point, det_delay)
    ax.plot(projectile[:, 0], projectile[:, 1], projectile[:, 2],
            'b--', label="抛体轨迹", linewidth=1.5)

```

```

# 烟雾轨迹和球体
if smoke_center is not None:
    smoke_path = compute_smoke_path(det_point, smoke_center)
    ax.plot(smoke_path[:, 0], smoke_path[:, 1], smoke_path[:, 2],
            'k--', label="烟雾轨迹", linewidth=1.5)
    ax.scatter(*smoke_center, c='orange', marker='x', s=100, label="烟雾中心")

# 绘制烟雾球体
u = np.linspace(0, 2 * np.pi, 30)
v = np.linspace(0, np.pi, 30)
x = smoke_center[0] + 10 * np.outer(np.cos(u), np.sin(v))
y = smoke_center[1] + 10 * np.outer(np.sin(u), np.sin(v))
z = smoke_center[2] + 10 * np.outer(np.ones(np.size(u)), np.cos(v))
ax.plot_surface(x, y, z, color='gray', alpha=0.5, rstride=1, cstride=1)

def plot_missile_path(ax, missile_config):
    """绘制导弹轨迹"""
    ax.plot([missile_config["start"][0], FAKE_TARGET[0]],
            [missile_config["start"][1], FAKE_TARGET[1]],
            [missile_config["start"][2], FAKE_TARGET[2]],
            'm--', label="导弹路径", linewidth=2)

def plot_target_cylinder(ax, target_spec):
    """绘制目标圆柱体"""
    theta = np.linspace(0, 2 * np.pi, 20)
    z = np.linspace(target_spec["center"][2], target_spec["center"][2] + target_spec["height"], 10)
    theta, z = np.meshgrid(theta, z)
    x = target_spec["center"][0] + target_spec["radius"] * np.cos(theta)
    y = target_spec["center"][1] + target_spec["radius"] * np.sin(theta)
    ax.plot_surface(x, y, z, alpha=0.3, color="cyan", rstride=1, cstride=1)

def setup_axes(ax, path, drop_point, det_point, smoke_center):
    """设置坐标轴属性"""
    ax.set_xlabel("X (米)")
    ax.set_ylabel("Y (米)")
    ax.set_zlabel("Z (米)")
    ax.set_box_aspect([1, 1, 1])
    ax.legend(loc='upper right', fontsize=8)
    ax.grid(True)

    points = np.vstack([path[-1:], [drop_point], [det_point]])
    if smoke_center is not None:
        points = np.vstack([points, [smoke_center]])

    margin = 50
    for i, set_lim in enumerate([ax.set_xlim, ax.set_ylim, ax.set_zlim]):
        min_val, max_val = np.min(points[:, i]), np.max(points[:, i])
        if min_val == max_val:
            min_val -= 10
            max_val += 10
        set_lim(min_val - margin, max_val + margin)

    ax.view_init(elev=30, azim=45)

def set_3d_equal_scale(ax):
    """设置 3D 坐标轴等比例"""
    x_lim = ax.get_xlim3d()
    y_lim = ax.get_ylim3d()
    z_lim = ax.get_zlim3d()
    x_range = abs(x_lim[1] - x_lim[0])
    y_range = abs(y_lim[1] - y_lim[0])
    z_range = abs(z_lim[1] - z_lim[0])
    max_range = max([x_range, y_range, z_range]) / 2.0

    mid_x = np.mean(x_lim)
    mid_y = np.mean(y_lim)
    mid_z = np.mean(z_lim)

    ax.set_xlim3d([mid_x - max_range, mid_x + max_range])
    ax.set_ylim3d([mid_y - max_range, mid_y + max_range])

```

```

ax.set_zlim3d([mid_z - max_range, mid_z + max_range])

def visualize_scene(uav_config, drop_point, det_point, missile_config, target_spec, smoke_center=None):
    """可视化整个场景"""
    validate_parameters(uav_config, missile_config, target_spec)

    fig = plt.figure(figsize=(10, 8))
    ax = fig.add_subplot(111, projection="3d")

    path = compute_uav_path(uav_config, drop_point)

    plot_uav_trajectory(ax, uav_config, drop_point)
    plot_key_points(ax, drop_point, det_point, smoke_center, uav_config["detonate_time"])
    plot_missile_path(ax, missile_config)
    plot_target_cylinder(ax, target_spec)

    setup_axes(ax, path, drop_point, det_point, smoke_center)
    set_3d_equal_scale(ax)
    plt.tight_layout()
    plt.show()

# ----- 6. 主程序 -----
if __name__ == "__main__":
    # 计算关键位置
    drop_pos = get_drop_point(
        uav_pos=UAV["start"],
        uav_speed=UAV["speed"],
        delay=UAV["drop_time"],
        target=FAKE_TARGET
    )

    det_pos = get_detonation_point(
        drop_pos=drop_pos,
        uav_speed=UAV["speed"],
        delay=UAV["detonate_time"],
        gravity=G,
        target=FAKE_TARGET
    )

    print("=== 关键位置 ===")
    print(f"无人机起始: {UAV['start'].round(4)}")
    print(f"烟幕投掷点: {drop_pos.round(4)}")
    print(f"烟幕引爆点: {det_pos.round(4)}")
    print(f"诱饵目标: {FAKE_TARGET}")

    # 生成目标采样点
    samples = create_target_samples(REAL_TARGET)
    print(f"n=== 采样信息 ===")
    print(f"目标采样点数量: {len(samples)}")

    # 计算导弹方向
    missile_vec = FAKE_TARGET - MISSILE["start"]
    missile_dist = np.linalg.norm(missile_vec)
    if missile_dist < EPS:
        missile_dir = np.array([0.0, 0.0, 0.0])
    else:
        missile_dir = missile_vec / missile_dist
    print(f"n=== 导弹信息 ===")
    print(f"导弹起始: {MISSILE['start'].round(4)}")
    print(f"导弹方向: {missile_dir.round(6)}")

    # 设置时间窗口
    det_time = UAV["drop_time"] + UAV["detonate_time"]
    start_time = det_time
    end_time = det_time + SMOKE["duration"]
    time_points = np.arange(start_time, end_time + DT, DT)
    print(f"n=== 时间范围 ===")
    print(f"烟幕引爆时间: {det_time:.2f}s")
    print(f"有效时间窗口: [{start_time:.2f}s, {end_time:.2f}s], 共 {len(time_points)}步")

```

```

# 时间步进模拟
total_cover_time = 0.0
cover_log = []
was_covered = False
cover_intervals = []

for t in time_points:
    # 计算导弹位置
    missile_pos = MISSILE["start"] + missile_dir * MISSILE["speed"] * t

    # 计算烟幕位置
    sink_time = t - det_time
    smoke_center = np.array([
        det_pos[0],
        det_pos[1],
        det_pos[2] - SMOKE["sink_speed"] * sink_time
    ])

    # 检查遮蔽状态
    covered = is_target_covered(missile_pos, smoke_center, SMOKE["radius"], samples)

    if covered:
        total_cover_time += DT
        cover_log.append({
            "time": round(t, 3),
            "missile": missile_pos.round(4),
            "smoke": smoke_center.round(4)
        })

    # 记录遮蔽区间
    if covered and not was_covered:
        cover_intervals.append({"start": t})
    elif not covered and was_covered:
        if cover_intervals:
            cover_intervals[-1]["end"] = t - DT

    was_covered = covered

if cover_intervals and "end" not in cover_intervals[-1]:
    cover_intervals[-1]["end"] = end_time

# 输出结果
print("\n" + "="*80)
print(f"【结果】 目标被遮蔽总时间: {total_cover_time:.4f} 秒")
print("="*80)

print("\n=== 遮蔽时间段 ===")
if not cover_intervals:
    print("无有效遮蔽")
else:
    for i, interval in enumerate(cover_intervals, 1):
        duration = interval["end"] - interval["start"]
        print(f"段{i}: {interval['start']:.4f}s ~ {interval['end']:.4f}s, 时长: {duration:.4f}s")

print("\n=== 遮蔽时刻示例 ===")
if cover_log:
    print("前 3 个时刻:")
    for log in cover_log[:3]:
        print(f"t={log['time']}s | 导弹: {log['missile']} | 烟幕: {log['smoke']}")

    print("\n最后 3 个时刻:")
    for log in cover_log[-3:]:
        print(f"t={log['time']}s | 导弹: {log['missile']} | 烟幕: {log['smoke']}")
else:
    print("无遮蔽时刻")

# 可视化
visualize_scene(UAV, drop_pos, det_pos, MISSILE, REAL_TARGET, smoke_center)

```

附录二：问题 2 代码求解

```
import numpy as np
import matplotlib.pyplot as plt
from joblib import Parallel, delayed
import multiprocessing
import time
import matplotlib as mpl

# 设置 matplotlib 的中文显示与美观样式
mpl.rcParams['font.sans-serif'] = ['SimHei']
mpl.rcParams['axes.unicode_minus'] = False

# ----- 1. 系统常量与参数定义 -----
GRAVITY = 9.81
EPSILON = 1e-15
TIME_STEP_COARSE = 0.1
TIME_STEP_FINE = 0.005
CPU_CORES = multiprocessing.cpu_count()

DECOY_TARGET = np.array([0.0, 0.0, 0.0])
REAL_TARGET = {
    "center": np.array([0.0, 200.0, 0.0]),
    "radius": 7.0,
    "height": 10.0
}

UAV_START = np.array([17800.0, 0.0, 1800.0])
SMOKE = {
    "radius": 10.0,
    "sink_speed": 3.0,
    "duration": 20.0
}
MISSILE = {
    "start": np.array([20000.0, 0.0, 2000.0]),
    "speed": 300.0
}
MISSILE_DIR = (DECOY_TARGET - MISSILE["start"]) / np.linalg.norm(DECOY_TARGET - MISSILE["start"])
MISSILE_TIME = np.linalg.norm(DECOY_TARGET - MISSILE["start"]) / MISSILE["speed"]

# ----- 2. 目标采样点生成 -----
def create_target_samples(target_spec):
    """生成目标表面和内部采样点，用于遮蔽检测"""
    points = []
    center = target_spec["center"]
    r, h = target_spec["radius"], target_spec["height"]
    center_xy = center[:2]
    bottom, top = center[2], center[2] + h

    # 生成角度和高度序列
    angles = np.linspace(0, 2*np.pi, 120, endpoint=False)
    heights = np.linspace(bottom, top, 40, endpoint=True)
    radii = np.linspace(0, r, 10, endpoint=True)
    z_inner = np.linspace(bottom, top, 30, endpoint=True)
    inner_angles = np.linspace(0, 2*np.pi, 24, endpoint=False)
    edge_radii = np.linspace(r*0.95, r*1.05, 5, endpoint=True)

    # 底面和顶面点
    for z in [bottom, top]:
        for angle in angles:
            x = center_xy[0] + r * np.cos(angle)
            y = center_xy[1] + r * np.sin(angle)
            points.append([x, y, z])

    # 侧面点
    for z in heights:
        for angle in angles:
            x = center_xy[0] + r * np.cos(angle)
            y = center_xy[1] + r * np.sin(angle)
            points.append([x, y, z])
```



```

# 内部点
for z in z_inner:
    for rad in radii:
        for angle in inner_angles:
            x = center_xy[0] + rad * np.cos(angle)
            y = center_xy[1] + rad * np.sin(angle)
            points.append([x, y, z])

# 边缘点
for z in np.linspace(bottom, top, 10):
    for rad in edge_radii:
        for angle in np.linspace(0, 2*np.pi, 60, endpoint=False):
            x = center_xy[0] + rad * np.cos(angle)
            y = center_xy[1] + rad * np.sin(angle)
            points.append([x, y, z])

return np.unique(np.array(points), axis=0)

# ----- 3. 几何计算与判定函数 -----
def vector_length(vector):
    """计算向量长度"""
    return np.sqrt(np.sum(vector**2))

def check_sphere_intersection(start, end, center, radius):
    """检查线段与球体是否相交，返回相交比例"""
    seg_vec = end - start
    center_vec = center - start
    a = np.dot(seg_vec, seg_vec)

    if a < EPSILON:
        return 1.0 if vector_length(center_vec) <= radius + EPSILON else 0.0

    b = -2 * np.dot(seg_vec, center_vec)
    c = np.dot(center_vec, center_vec) - radius**2
    discriminant = b**2 - 4*a*c

    if discriminant < -EPSILON:
        return 0.0

    if discriminant < 0:
        discriminant = 0.0

    sqrt_d = np.sqrt(discriminant)
    s1 = (-b - sqrt_d) / (2*a)
    s2 = (-b + sqrt_d) / (2*a)
    s_start = max(0.0, min(s1, s2))
    s_end = min(1.0, max(s1, s2))

    return max(0.0, s_end - s_start)

def is_fully_covered(missile_pos, smoke_center, smoke_radius, samples):
    """检查目标是否被完全遮蔽"""
    for point in samples:
        if check_sphere_intersection(missile_pos, point, smoke_center, smoke_radius) < EPSILON:
            return False
    return True

# ----- 4. 自适应时间步长生成 -----
def create_time_steps(start, end, event_time=None):
    """生成自适应时间序列，关键事件附近使用更小步长"""
    if event_time is None:
        return np.arange(start, end + TIME_STEP_COARSE, TIME_STEP_COARSE)

    fine_start = max(start, event_time - 1.0)
    fine_end = min(end, event_time + 1.0)
    times = []

    if start < fine_start:
        times.extend(np.arange(start, fine_start, TIME_STEP_COARSE))

```

```

times.extend(np.arange(fine_start, fine_end + TIME_STEP_FINE, TIME_STEP_FINE))

if fine_end < end:
    times.extend(np.arange(fine_end, end + TIME_STEP_COARSE, TIME_STEP_COARSE))

return np.unique(times)

# ----- 5. 适应度函数 -----
def evaluate_fitness(params, samples):
    """评估无人机投放策略的有效性，返回遮蔽时长"""
    angle, speed, drop_delay, det_delay = params
    direction = np.array([np.cos(angle), np.sin(angle), 0.0])

    # 计算投掷点和引爆点
    drop_pos = UAV_START + speed * drop_delay * direction
    det_xy = drop_pos[:2] + speed * det_delay * direction[:2]
    det_z = drop_pos[2] - 0.5 * GRAVITY * det_delay**2

    # 检查引爆高度是否过低
    if det_z < 5.0:
        return 0.0 + np.random.uniform(-0.5, 0)

    det_pos = np.array([det_xy[0], det_xy[1], det_z])
    det_time = drop_delay + det_delay

    # 计算导弹到达时间
    target_vec = REAL_TARGET["center"] - MISSILE["start"]
    dist_proj = np.dot(target_vec, MISSILE_DIR)
    event_time = dist_proj / MISSILE["speed"]

    # 确定时间范围
    smoke_end = det_time + SMOKE["duration"]
    end_time = min(smoke_end, MISSILE_TIME)

    if det_time >= end_time:
        return 0.0 + np.random.uniform(-0.1, 0)

    # 时间步进模拟
    time_points = create_time_steps(det_time, end_time, event_time)
    cover_time = 0.0
    prev_t = None

    for t in time_points:
        if prev_t is not None:
            dt = t - prev_t
            missile_pos = MISSILE["start"] + MISSILE["speed"] * t * MISSILE_DIR
            sink_time = t - det_time
            smoke_z = det_pos[2] - SMOKE["sink_speed"] * sink_time

            # 检查烟幕高度
            if smoke_z < 2.0:
                prev_t = t
                continue

            smoke_center = np.array([det_pos[0], det_pos[1], smoke_z])

            if is_fully_covered(missile_pos, smoke_center, SMOKE["radius"], samples):
                cover_time += dt

            prev_t = t

    # 边界奖励
    boundary_bonus = 0.0
    if abs(speed - 70) < 1 or abs(speed - 140) < 1:
        boundary_bonus = 0.1
    if drop_delay < 1 or det_delay < 1:
        boundary_bonus += 0.1

    return cover_time + boundary_bonus

```

```

# ----- 6. 粒子群优化算法 -----
class PSO:
    def __init__(self, objective_func, bounds, num_particles=30, max_iter=100,
                  c1=2.0, c2=2.0, w_start=0.9, w_end=0.4):
        self.objective = objective_func
        self.bounds = bounds
        self.num_particles = num_particles
        self.max_iter = max_iter
        self.c1 = c1
        self.c2 = c2
        self.w_start = w_start
        self.w_end = w_end
        self.dim = len(bounds)
        self.positions = np.zeros((num_particles, self.dim))
        self.velocities = np.zeros((num_particles, self.dim))
        self.pbest_pos = np.zeros((num_particles, self.dim))
        self.pbest_fit = np.zeros(num_particles) - np.inf
        self.gbest_pos = np.zeros(self.dim)
        self.gbest_fit = -np.inf
        self.history = []
        self._init_particles()

    def _init_particles(self):
        """初始化粒子群"""
        for i in range(self.num_particles):
            for j in range(self.dim):
                low, high = self.bounds[j]
                self.positions[i, j] = np.random.uniform(low, high)
                range_val = high - low
                self.velocities[i, j] = np.random.uniform(-0.1*range_val, 0.1*range_val)

            fitness = self.objective(self.positions[i])
            self.pbest_pos[i] = self.positions[i].copy()
            self.pbest_fit[i] = fitness

            if fitness > self.gbest_fit:
                self.gbest_fit = fitness
                self.gbest_pos = self.positions[i].copy()

    def _constrain_pos(self, pos, dim):
        """约束位置在边界内"""
        low, high = self.bounds[dim]
        if pos < low:
            return low + 0.01 * (np.random.random() - 0.5)
        elif pos > high:
            return high + 0.01 * (np.random.random() - 0.5)
        return pos

    def _constrain_vel(self, vel, dim):
        """约束速度大小"""
        low, high = self.bounds[dim]
        limit = 0.2 * (high - low)
        return np.clip(vel, -limit, limit)

    def optimize(self):
        """执行优化过程"""
        for iter in range(self.max_iter):
            mean_fit = np.mean(self.pbest_fit)
            w = self.w_start - (self.w_start - self.w_end) * (iter / self.max_iter) + (1 - self.w_end) * mean_fit /
            np.max(self.pbest_fit)

            # 并行计算适应度
            fitness_vals = Parallel(n_jobs=CPU_CORES)(
                delayed(self.objective)(self.positions[i])
                for i in range(self.num_particles)
            )

            for i in range(self.num_particles):
                fit = fitness_vals[i]

```

```

        if fit > self.pbest_fit[i]:
            self.pbest_fit[i] = fit
            self.pbest_pos[i] = self.positions[i].copy()

        if fit > self.gbest_fit:
            self.gbest_fit = fit
            self.gbest_pos = self.positions[i].copy()

        # 更新速度和位置
        r1 = np.random.random(self.dim)
        r2 = np.random.random(self.dim)
        cognitive = self.c1 * r1 * (self.pbest_pos[i] - self.positions[i])
        social = self.c2 * r2 * (self.gbest_pos - self.positions[i])

        new_vel = w * self.velocities[i] + cognitive + social

        for j in range(self.dim):
            new_vel[j] = self._constrain_vel(new_vel[j], j)

        self.velocities[i] = new_vel
        new_pos = self.positions[i] + new_vel

        for j in range(self.dim):
            new_pos[j] = self._constrain_pos(new_pos[j], j)

        self.positions[i] = new_pos

    self.history.append(self.gbest_fit)

    if (iter + 1) % 10 == 0 or iter == 0:
        print(f'迭代 {iter+1}/{self.max_iter}, 当前最优适应度: {self.gbest_fit:.6f}')

    return self.gbest_pos, self.gbest_fit, self.history

# ----- 7. 主程序 -----
if __name__ == "__main__":
    start_time = time.time()

    print("正在生成目标采样点...")
    target_samples = create_target_samples(REAL_TARGET)
    print(f'采样点总数: {len(target_samples)}')

    # 参数边界
    bounds = [
        (0.0, 2 * np.pi),    # 方向角
        (70.0, 140.0),        # 无人机速度
        (0.0, 80.0),          # 投掷延迟
        (0.0, 25.0)           # 引爆延迟
    ]

    def objective(params):
        return evaluate_fitness(params, target_samples)

    print("\n 开始执行粒子群优化...")
    pso = PSO(
        objective_func=objective,
        bounds=bounds,
        num_particles=50,
        max_iter=100,
        c1=1.5,
        c2=1.5,
        w_start=0.9,
        w_end=0.4
    )

    best_params, best_fitness, history = pso.optimize()

    # 解析最优参数
    best_angle, best_speed, best_drop_delay, best_det_delay = best_params

```

```

best_dir = np.array([np.cos(best_angle), np.sin(best_angle), 0.0])
best_drop_pos = UAV_START + best_speed * best_drop_delay * best_dir
best_det_xy = best_drop_pos[2:] + best_speed * best_det_delay * best_dir[2:]
best_det_z = best_drop_pos[2] - 0.5 * GRAVITY * best_det_delay**2
best_det_pos = np.array([best_det_xy[0], best_det_xy[1], best_det_z])
best_det_time = best_drop_delay + best_det_delay

# 验证最优解
verify_fitness = evaluate_fitness(best_params, target_samples)
end_time = time.time()
elapsed_time = end_time - start_time

# 输出优化结果
print("\n" + "="*80)
print("粒子群优化结果报告".center(80))
print("="*80)
print(f"\n[ 性能分析 ]")
print(f" - 总耗时: {elapsed_time:.2f} 秒")
print(f" - 最佳适应度 (有效遮蔽时长): {best_fitness:.4f} 秒")
print(f" - 验证阶段适应度: {verify_fitness:.4f} 秒")

print(f"\n[ 最佳投放策略参数 ]")
print(f" - 无人机速度: {best_speed:.2f} m/s")
print(f" - 飞行方向角: {np.degrees(best_angle):.2f}° ({best_angle:.4f} rad)")
print(f" - 投放延迟时间: {best_drop_delay:.2f} s")
print(f" - 起爆延迟时间: {best_det_delay:.2f} s")

print(f"\n[ 关键行动点坐标 ]")
print(f" - 投放点坐标: ({best_drop_pos[0]:.2f}, {best_drop_pos[1]:.2f}, {best_drop_pos[2]:.2f})")
print(f" - 起爆点坐标: ({best_det_pos[0]:.2f}, {best_det_pos[1]:.2f}, {best_det_pos[2]:.2f})")

print(f"\n[ 烟幕生效时间 ]")
print(f" - 起爆时间: {best_det_time:.2f} s")
print(f" - 结束时间: {best_det_time + SMOKE['duration']:.2f} s")
print("="*80)

# 绘制优化收敛曲线
plt.figure(figsize=(10, 6))
plt.plot(history)
plt.title('粒子群优化收敛曲线')
plt.xlabel('迭代次数')
plt.ylabel('最优遮蔽时长 (s)')
plt.grid(True)
plt.savefig('pso_convergence.png')
plt.show()

```

附录三：问题 3 代码求解

```

import numpy as np
import random
import matplotlib.pyplot as plt
from joblib import Parallel, delayed
import multiprocessing
import pandas as pd
import matplotlib as mpl

# --- 全局配置 ---
# 设置绘图支持中文显示
mpl.rcParams['font.sans-serif'] = ['SimHei']
mpl.rcParams['axes.unicode_minus'] = False

# 定义全局常量
GRAVITY_ACCEL = 9.81 # 重力加速度 (m/s²)
NUMERIC_THRESHOLD = 1e-15 # 数值运算的保护阈值
COARSE_TIME_STEP = 0.1 # 粗略计算时的时间步长
FINE_TIME_STEP = 0.005 # 关键时段的精细步长
MIN_DROP_INTERVAL = 1.0 # 投放烟幕弹的最小时间间隔
PARALLEL_JOBS = multiprocessing.cpu_count() # 并行计算使用的 CPU 核心数

```

```

# 定义目标参数
FALSE_TARGET_POS = np.array([0.0, 0.0, 0.0]) # 假目标的坐标
TRUE_TARGET_SPEC = {
    "base_center": np.array([0.0, 200.0, 0.0]), # 真目标的底面圆心
    "radius": 7.0, # 真目标圆柱体的半径
    "height": 10.0 # 真目标圆柱体的高度
}

# 定义烟幕参数
SMOKE_CONFIG = {
    "effective_radius": 10.0, # 烟幕的有效遮蔽半径 (m)
    "sink_velocity": 3.0, # 烟幕下沉的速度 (m/s)
    "active_duration": 20.0 # 烟幕的有效遮蔽时间 (s)
}

# 定义无人机和导弹参数
UAV_INITIAL_POS = np.array([17800.0, 0.0, 1800.0]) # 无人机的初始位置
MISSILE_CONFIG = {
    "initial_pos": np.array([20000.0, 0.0, 2000.0]), # 导弹的初始位置
    "flight_speed": 300.0 # 导弹的飞行速度 (m/s)
}

# 计算导弹的飞行方向和到达假目标的时间
MISSILE_DIRECTION = (FALSE_TARGET_POS - MISSILE_CONFIG["initial_pos"]) /
np.linalg.norm(FALSE_TARGET_POS - MISSILE_CONFIG["initial_pos"])
MISSILE_ARRIVAL_TIME = np.linalg.norm(FALSE_TARGET_POS - MISSILE_CONFIG["initial_pos"]) /
MISSILE_CONFIG["flight_speed"]

# --- 工具函数（未作改动） ---
def vector_magnitude(vector):
    """计算向量的模长。"""
    return np.sqrt(np.sum(vector**2))

def segment_sphere_intersection(start_point, end_point, sphere_center, sphere_radius):
    """判定高精度的线段与球体相交情况。"""
    segment_vec = end_point - start_point
    sphere_vec = sphere_center - start_point
    a = np.dot(segment_vec, segment_vec)
    if a < NUMERIC_THRESHOLD:
        dist = vector_magnitude(sphere_vec)
        return 1.0 if dist <= sphere_radius + NUMERIC_THRESHOLD else 0.0
    b = -2 * np.dot(segment_vec, sphere_vec)
    c = np.dot(sphere_vec, sphere_vec) - sphere_radius**2
    discriminant = b**2 - 4*a*c
    if discriminant < -NUMERIC_THRESHOLD:
        return 0.0
    if discriminant < 0:
        discriminant = 0.0
    sqrt_d = np.sqrt(discriminant)
    s1 = (-b - sqrt_d) / (2*a)
    s2 = (-b + sqrt_d) / (2*a)
    s_start = max(0.0, min(s1, s2))
    s_end = min(1.0, max(s1, s2))
    return max(0.0, s_end - s_start)

def is_fully_shielded_multi(missile_pos, smoke_centers, smoke_radius, target_samples):
    """判定真目标是否被多个烟幕完全遮蔽。"""
    for sample in target_samples:
        shielded = False
        for smoke_center in smoke_centers:
            if segment_sphere_intersection(missile_pos, sample, smoke_center, smoke_radius) >=
NUMERIC_THRESHOLD:
                shielded = True
                break
        if not shielded:
            return False
    return True

def get_adaptive_time_steps(start_time, end_time, event_time=None):
    """生成自适应的时间步长序列。"""

```

```

if event_time is None:
    return np.arange(start_time, end_time + COARSE_TIME_STEP, COARSE_TIME_STEP)
fine_start = max(start_time, event_time - 1.0)
fine_end = min(end_time, event_time + 1.0)
times = []
if start_time < fine_start:
    times.extend(np.arange(start_time, fine_start, COARSE_TIME_STEP))
times.extend(np.arange(fine_start, fine_end + FINE_TIME_STEP, FINE_TIME_STEP))
if fine_end < end_time:
    times.extend(np.arange(fine_end, end_time + COARSE_TIME_STEP, COARSE_TIME_STEP))
return np.unique(times)

# --- 目标采样点生成（未作改动） ---
def generate_dense_samples(target_spec):
    """生成真目标的超高密度采样点。"""
    samples = []
    center, radius, height = target_spec["base_center"], target_spec["radius"], target_spec["height"]
    center_xy = center[:2]
    min_z, max_z = center[2], center[2] + height
    theta_dense = np.linspace(0, 2*np.pi, 120, endpoint=False)
    for z in [min_z, max_z]:
        for th in theta_dense:
            x = center_xy[0] + radius * np.cos(th)
            y = center_xy[1] + radius * np.sin(th)
            samples.append([x, y, z])
    heights_dense = np.linspace(min_z, max_z, 40, endpoint=True)
    for z in heights_dense:
        for th in theta_dense:
            x = center_xy[0] + radius * np.cos(th)
            y = center_xy[1] + radius * np.sin(th)
            samples.append([x, y, z])
    radii = np.linspace(0, radius, 10, endpoint=True)
    inner_heights = np.linspace(min_z, max_z, 30, endpoint=True)
    inner_thetas = np.linspace(0, 2*np.pi, 24, endpoint=False)
    for z in inner_heights:
        for rad in radii:
            for th in inner_thetas:
                x = center_xy[0] + rad * np.cos(th)
                y = center_xy[1] + rad * np.sin(th)
                samples.append([x, y, z])
    edge_radii = np.linspace(radius*0.95, radius*1.05, 5, endpoint=True)
    for z in np.linspace(min_z, max_z, 10):
        for rad in edge_radii:
            for th in np.linspace(0, 2*np.pi, 60, endpoint=False):
                x = center_xy[0] + rad * np.cos(th)
                y = center_xy[1] + rad * np.sin(th)
                samples.append([x, y, z])
    return np.unique(np.array(samples), axis=0)

# --- 适应度函数（未作改动） ---
def fitness_v_t_intervals(params, debug=False):
    """
    计算适应度，其值为距离加权和的负数。
    目的是寻找使烟幕起爆点与导弹轨迹距离最短的参数组合。
    """
    uav_speed, drop_delay_1, drop_delay_2, drop_delay_3 = params
    if not (70.0 <= uav_speed <= 140.0) or any(t < MIN_DROP_INTERVAL for t in (drop_delay_1, drop_delay_2, drop_delay_3)):
        return -1e6
    total_time = drop_delay_1 + drop_delay_2 + drop_delay_3
    total_time_p = total_time + uav_speed * drop_delay_2 / 298.5
    total_time_pp = total_time + uav_speed * (drop_delay_1 + drop_delay_2) / 298.5
    point_a = np.array([17800.0 + uav_speed * (drop_delay_1 + drop_delay_2), 0.0, 1800.0 - 3.0 * drop_delay_3])
    point_b = np.array([17800.0 + uav_speed * drop_delay_1, 0.0, 1800.0 - 3.0 * (drop_delay_3 + drop_delay_2 + uav_speed * drop_delay_2 / 298.5)])
    point_c = np.array([17800.0, 0.0, 1800.0 - 3.0 * total_time_pp])
    missile_pos_1 = np.array([20000.0 - 298.5 * total_time, 0.0, 2000.0 - 29.85 * total_time])
    missile_pos_2 = np.array([20000.0 - 298.5 * total_time_p, 0.0, 2000.0 - 29.85 * total_time_p])
    missile_pos_3 = np.array([20000.0 - 298.5 * total_time_pp, 0.0, 2000.0 - 29.85 * total_time_pp])
    dist_a = np.linalg.norm(point_a - missile_pos_1)

```

```

dist_b = np.linalg.norm(point_b - missile_pos_2)
dist_c = np.linalg.norm(point_c - missile_pos_3)
weighted_dist_sum = 15*dist_a + 0.3*dist_b + 0.1*dist_c
penalty = 0.0
z_vals = [point_a[2], point_b[2], point_c[2], missile_pos_1[2], missile_pos_2[2], missile_pos_3[2]]
if any(z < 0 for z in z_vals):
    penalty += 1e4 + 1e3 * sum(max(0.0, -z) for z in z_vals)
if not np.isfinite(weighted_dist_sum):
    penalty += 1e6
fitness = -weighted_dist_sum - penalty
if debug:
    print(f"[调试] dist_a={dist_a:.2f}, dist_b={dist_b:.2f}, dist_c={dist_c:.2f}, "
          f"weighted_dist_sum={weighted_dist_sum:.2f}, penalty={penalty:.2f}, fitness={fitness:.2f}")
return fitness

# --- 遮蔽时间计算函数（已更新） ---
def compute_shield_duration(params, target_samples):
    """
    计算各起爆点产生的烟幕的遮蔽时间以及总的有效遮蔽时长。
    该函数包含详细的调试输出。
    """
    uav_speed, drop_delay_1, drop_delay_2, drop_delay_3 = params
    total_time = drop_delay_1 + drop_delay_2 + drop_delay_3
    total_time_p = total_time + uav_speed * drop_delay_2 / 298.5
    total_time_pp = total_time + uav_speed * (drop_delay_1 + drop_delay_2) / 298.5

    if not (70.0 <= uav_speed <= 140.0) or any(t < 0 for t in [drop_delay_1, drop_delay_2, drop_delay_3]):
        print(f"[调试] 参数无效: uav_speed={uav_speed:.2f}, drop_delay_1={drop_delay_1:.2f}, "
              f"drop_delay_2={drop_delay_2:.2f}, drop_delay_3={drop_delay_3:.2f}")
        return 0.0, [0.0, 0.0, 0.0], []

    # 计算烟幕起爆点和对应时刻的导弹位置
    explosion_points = [
        np.array([17800.0 + uav_speed * (drop_delay_1 + drop_delay_2), 0.0, 1800.0 - 3.0 * drop_delay_3]),
        np.array([17800.0 + uav_speed * drop_delay_1, 0.0, 1800.0 - 3.0 * (drop_delay_3 + drop_delay_2 + uav_speed *
drop_delay_2 / 298.5)]),
        np.array([17800.0, 0.0, 1800.0 - 3.0 * total_time_pp])
    ]
    detonation_times = [drop_delay_1, drop_delay_1 + drop_delay_2, drop_delay_1 + drop_delay_2 + drop_delay_3]
    explosion_labels = ['A', 'B', 'C']

    # 调试: 输出计算出的坐标
    print("[调试] 烟幕起爆点坐标: ")
    for label, point in zip(explosion_labels, explosion_points):
        print(f"{label}: {point.round(4)}")
    print(f"导弹位置:\n M1: {np.array([20000.0 - 298.5 * total_time, 0.0, 2000.0 - 29.85 * total_time]).round(4)}\n M2:
{np.array([20000.0 - 298.5 * total_time_p, 0.0, 2000.0 - 29.85 * total_time_p]).round(4)}\n M3: {np.array([20000.0 - 298.5
* total_time_pp, 0.0, 2000.0 - 29.85 * total_time_pp]).round(4)}")
    dist_a = np.linalg.norm(explosion_points[0] - np.array([20000.0 - 298.5 * total_time, 0.0, 2000.0 - 29.85 * total_time]))
    dist_b = np.linalg.norm(explosion_points[1] - np.array([20000.0 - 298.5 * total_time_p, 0.0, 2000.0 - 29.85 *
total_time_p]))
    dist_c = np.linalg.norm(explosion_points[2] - np.array([20000.0 - 298.5 * total_time_pp, 0.0, 2000.0 - 29.85 *
total_time_pp]))
    print(f"距离:\n dist_a: {dist_a:.4f}\n dist_b: {dist_b:.4f}\n dist_c: {dist_c:.4f}")

    # 定义仿真时间范围
    start_time = min(detonation_times)
    smoke_end_times = [t_det + SMOKE_CONFIG["active_duration"] for t_det in detonation_times]
    end_time = min(max(smoke_end_times), MISSILE_ARRIVAL_TIME)
    if start_time >= end_time:
        print(f"[调试] 时间范围无效: start_time={start_time:.2f}, end_time={end_time:.2f}")
        return 0.0, [0.0, 0.0, 0.0], []

    # 计算导弹到达真目标投影点的时间
    missile_to_target = TRUE_TARGET_SPEC["base_center"] - MISSILE_CONFIG["initial_pos"]
    dist_proj = np.dot(missile_to_target, MISSILE_DIRECTION)
    event_time = dist_proj / MISSILE_CONFIG["flight_speed"]

    # 生成自适应时间步长序列
    time_steps = get_adaptive_time_steps(start_time, end_time, event_time)

```



```

# 初始化遮蔽记录
total_shield_duration = 0.0
shield_durations = [0.0, 0.0, 0.0]
shield_intervals = []
was_shielded = False

# 调试：输出时间范围
print(f"[调试] 时间范围: start_time={start_time:.2f}, end_time={end_time:.2f}, event_time={event_time:.2f}")
print("[调试] 开始时间步循环...")

# 时间步循环
prev_time = None
for t in time_steps:
    if prev_time is not None:
        dt = t - prev_time
        missile_pos = MISSILE_CONFIG["initial_pos"] + MISSILE_CONFIG["flight_speed"] * t *
MISSILE_DIRECTION

        # 确定当前时刻哪些烟幕是活跃的
        active_smoke_centers = []
        active_indices = []
        for i, t_det in enumerate(detonation_times):
            if t_det <= t < t_det + SMOKE_CONFIG["active_duration"]:
                sink_time = t - t_det
                smoke_z = explosion_points[i][2] - SMOKE_CONFIG["sink_velocity"] * sink_time
                if smoke_z < 2.0:
                    continue
                smoke_center = np.array([explosion_points[i][0], explosion_points[i][1], smoke_z])
                active_smoke_centers.append(smoke_center)
                active_indices.append(i)

        # 判定是否被整体遮蔽
        current_shielded = False
        if active_smoke_centers:
            current_shielded = is_fully_shielded_multi(missile_pos, active_smoke_centers,
SMOKE_CONFIG["effective_radius"], target_samples)

        # 记录总遮蔽时长
        if current_shielded:
            total_shield_duration += dt

        # 记录每个烟幕的遮蔽时长
        for i, t_det in enumerate(detonation_times):
            if t_det <= t < t_det + SMOKE_CONFIG["active_duration"]:
                smoke_z = explosion_points[i][2] - SMOKE_CONFIG["sink_velocity"] * (t - t_det)
                if smoke_z < 2.0:
                    continue
                smoke_center = np.array([explosion_points[i][0], explosion_points[i][1], smoke_z])
                if is_fully_shielded_multi(missile_pos, [smoke_center], SMOKE_CONFIG["effective_radius"],
target_samples):
                    shield_durations[i] += dt

        # 记录遮蔽时间段
        if current_shielded and not was_shielded:
            shield_intervals.append({"start": t})
        elif not current_shielded and was_shielded:
            if shield_intervals:
                shield_intervals[-1]["end"] = t - dt

        was_shielded = current_shielded
        prev_time = t

# 处理最后一个未结束的遮蔽时间段
if shield_intervals and "end" not in shield_intervals[-1]:
    shield_intervals[-1]["end"] = end_time

return total_shield_duration, shield_durations, shield_intervals

# --- 粒子群优化器（未作改动） ---

```

```

class ParticleSwarmOptimizer:
    def __init__(self, objective_func, bounds, num_particles=50, max_iter=100,
                  c1=2.0, c2=2.0, w_start=0.9, w_end=0.4):
        """初始化粒子群优化器。"""
        self.objective_func = objective_func
        self.bounds = bounds
        self.num_particles = num_particles
        self.max_iter = max_iter
        self.c1, self.c2 = c1, c2
        self.w_start, self.w_end = w_start, w_end
        self.dim = len(bounds)
        self.positions = np.array([
            [random.uniform(bounds[d][0], bounds[d][1]) for d in range(self.dim)]
            for _ in range(num_particles)
        ])
        self.velocities = np.random.uniform(-1, 1, (num_particles, self.dim))
        self.personal_best_positions = np.copy(self.positions)
        self.personal_best_scores = np.array([float('-inf')] * num_particles)
        self.global_best_position = None
        self.global_best_score = float('-inf')
        self.history = []

    def optimize(self):
        """执行粒子群优化过程。"""
        for gen in range(self.max_iter):
            w = self.w_start - (self.w_start - self.w_end) * (gen / self.max_iter)
            scores = Parallel(n_jobs=PARALLEL_JOBS)(
                delayed(self.objective_func)(particle) for particle in self.positions
            )
            for i, score in enumerate(scores):
                if score > self.personal_best_scores[i]:
                    self.personal_best_scores[i] = score
                    self.personal_best_positions[i] = self.positions[i]
                if score > self.global_best_score:
                    self.global_best_score = score
                    self.global_best_position = self.positions[i]
            for i in range(self.num_particles):
                r1, r2 = random.random(), random.random()
                cognitive = self.c1 * r1 * (self.personal_best_positions[i] - self.positions[i])
                social = self.c2 * r2 * (self.global_best_position - self.positions[i])
                self.velocities[i] = w * self.velocities[i] + cognitive + social
                self.positions[i] += self.velocities[i]
                for d in range(self.dim):
                    if self.positions[i][d] < self.bounds[d][0]:
                        self.positions[i][d] = self.bounds[d][0]
                        self.velocities[i][d] *= -0.5
                    elif self.positions[i][d] > self.bounds[d][1]:
                        self.positions[i][d] = self.bounds[d][1]
                        self.velocities[i][d] *= -0.5
            self.history.append(self.global_best_score)
            if gen % 10 == 0:
                uav_speed, drop_delay_1, drop_delay_2, drop_delay_3 = self.global_best_position
                print(f"迭代 {gen}, 最优适应度 = {self.global_best_score:.6f}")
                print(f"参数: uav_speed={uav_speed:.2f}, drop_delay_1={drop_delay_1:.2f},
drop_delay_2={drop_delay_2:.2f}, drop_delay_3={drop_delay_3:.2f}")
                fitness_v_t_intervals(self.global_best_position, debug=True)
            return self.global_best_position, self.global_best_score, self.history

# --- 主程序（已更新） ---
if __name__ == "__main__":
    # 生成目标采样点
    print("生成目标采样点...")
    target_samples = generate_dense_samples(TRUE_TARGET_SPEC)
    print(f"采样点数量: {len(target_samples)}")

    # 设置优化参数的边界
    bounds = [
        (70.0, 140.0), # 无人机速度
        (1.0, 80.0), # 第一次投放延迟
        (1.0, 80.0), # 第二次投放延迟
    ]

```

```

    (0, 80.0)      # 第三次投放延迟
]

# 初始化并运行 PSO
pso = ParticleSwarmOptimizer(
    objective_func=fitness_v_t_intervals,
    bounds=bounds,
    num_particles=60,
    max_iter=150,
    c1=1.5, c2=1.5,
    w_start=0.9, w_end=0.4
)
best_params, best_fitness, history = pso.optimize()

# 使用最优参数计算遮蔽时间
opt_uav_speed, opt_drop_delay_1, opt_drop_delay_2, opt_drop_delay_3 = best_params
min_weighted_dist_sum = -best_fitness if np.isfinite(best_fitness) else None
total_shield_duration, shield_durations, shield_intervals = compute_shield_duration(best_params, target_samples)

# --- 最终结果输出 ---
print("\n" + "="*80)
print("优化结果")
print("="*80)

# 显示最优参数和目标函数值
print(f"目标函数值 (最小化加权距离和): {min_weighted_dist_sum:.6f}")
print(f"找到的最优参数:")
print(f"  无人机速度: {opt_uav_speed:.3f} 米/秒")
print(f"  第一次投放延迟: {opt_drop_delay_1:.3f} 秒")
print(f"  第二次投放延迟: {opt_drop_delay_2:.3f} 秒")
print(f"  第三次投放延迟: {opt_drop_delay_3:.3f} 秒")

# 显示遮蔽时长分析
print("\n 遮蔽时长分析:")
print(f"总遮蔽时长: {total_shield_duration:.4f} 秒")
print("各烟幕单点遮蔽时长:")
for i, duration in enumerate(shield_durations):
    print(f"  起爆点  {['A', 'B', 'C'][i]}: {duration:.4f} 秒")

# 显示遮蔽时间段
print("\n 遮蔽时间段详情:")
if not shield_intervals:
    print("  未找到有效的遮蔽时间段。")
else:
    for i, interval in enumerate(shield_intervals, 1):
        duration = interval["end"] - interval["start"]
        print(f"  第{i}段: {interval['start']:.4f}秒 ~ {interval['end']:.4f}秒 (时长: {duration:.4f}秒)")
print("="*80)

# 绘制收敛曲线
plt.figure(figsize=(10, 6))
plt.plot(history, label="最优适应度")
plt.xlabel("迭代次数")
plt.ylabel("适应度")
plt.title("PSO 收敛曲线")
plt.legend()
plt.grid(True)
plt.show()

```

附录四：问题 4 代码求解

```

from itertools import combinations
import numpy as np
from scipy.optimize import minimize
import random

# --- 无人机与导弹配置 ---
# 定义 5 架无人机的初始位置
UAV_INITIAL_POSITIONS = {

```

```

    "UAV1": np.array([17800.0, 0.0, 1800.0]),
    "UAV2": np.array([12000.0, 1400.0, 1400.0]),
    "UAV3": np.array([6000.0, -3000.0, 700.0]),
    "UAV4": np.array([11000.0, 2000.0, 1800.0]),
    "UAV5": np.array([13000.0, -2000.0, 1300.0]),
}
UAV_IDS = list(UAV_INITIAL_POSITIONS.keys())

# 定义单个导弹的配置
MISSILE_CONFIGS = [
    {"id": "M1", "initial_pos": np.array([20000.0, 0.0, 2000.0]), "speed": 300.0},
]

# 无人机的基础覆盖能力矩阵（这里只用第一列数据）
COVERAGE_MATRIX_FULL = np.array([
    [4.6, 7.686, 1.246],
    [3.08, 3.9, 1.746],
    [2.92, 2.471, 3.1],
    [3.501, 1.212, 0.989],
    [1.877, 3.215, 0.36]
])
COVERAGE_MATRIX = COVERAGE_MATRIX_FULL[:, 0].reshape(-1, 1)

# 全局优化参数
MAX_DISTANCE = 10000.0 # 无人机最大移动距离
WOA_POPULATION_SIZE = 30 # 鲸鱼优化算法的种群大小
WOA_ITERATIONS = 100 # 鲸鱼优化算法的迭代次数
RANDOM_SEED = 42 # 随机种子，用于结果复现
random.seed(RANDOM_SEED)
np.random.seed(RANDOM_SEED)

# --- 适应度函数 ---
def compute_coverage(uav_index, uav_position, missile_position):
    """根据距离和基础覆盖矩阵计算无人机的实际覆盖能力。"""
    distance = np.linalg.norm(uav_position - missile_position)
    # 覆盖能力随距离指数衰减
    return COVERAGE_MATRIX[uav_index, 0] * np.exp(-distance / 5000.0)

def fitness_function_with_individual(positions, selected_indices):
    """
    计算给定无人机位置的总覆盖能力及个体贡献。
    目标是最大化总覆盖能力，因此返回其负值。
    """
    total_coverage = 0.0
    individual_coverages = []
    for i, uav_idx in enumerate(selected_indices):
        uav_pos = positions[3*i:3*i+3]
        # 计算无人机与其初始位置的距离
        dist_from_initial = np.linalg.norm(uav_pos - UAV_INITIAL_POSITIONS[UAV_IDS[uav_idx]])
        # 限制无人机的最大移动距离
        scale = min(1.0, MAX_DISTANCE / (dist_from_initial + 1e-6))
        coverage = compute_coverage(uav_idx, uav_pos, MISSILE_CONFIGS[0]["initial_pos"]) * scale
        individual_coverages.append(coverage)
        total_coverage += coverage
    return -total_coverage, individual_coverages

# --- 鲸鱼优化算法 (WOA) ---
def initialize_population(pop_size, num_uavs):
    """在指定范围内初始化种群位置。"""
    pos_min, pos_max = [], []
    for uav_idx in range(num_uavs):
        for dim in range(3):
            # 搜索范围以初始位置为中心，最大移动距离为半径
            pos_min.append(UAV_INITIAL_POSITIONS[UAV_IDS[uav_idx]][dim] - MAX_DISTANCE)
            pos_max.append(UAV_INITIAL_POSITIONS[UAV_IDS[uav_idx]][dim] + MAX_DISTANCE)
    population = np.random.uniform(low=pos_min, high=pos_max, size=(pop_size, 3*num_uavs))
    return population, np.array(pos_min), np.array(pos_max)

def woa_update_population(population, best_position, decay_factor, pos_min, pos_max):
    """使用鲸鱼优化算法更新种群位置。"""

```

```

new_population = np.zeros_like(population)
for i in range(population.shape[0]):
    r1, r2 = np.random.rand(), np.random.rand()
    A = 2 * decay_factor * r1 - decay_factor
    C = 2 * r2
    D = np.abs(C * best_position - population[i])
    new_population[i] = best_position - A * D
    # 确保新位置不超出界限
    new_population[i] = np.minimum(np.maximum(new_population[i], pos_min), pos_max)
return new_population

# --- 主优化循环：枚举组合并进行优化 ---
total_uav_count = len(UAV_IDS)
uav_selection_count = 3 # 从5架中选择3架
optimization_results = []

print("开始枚举无人机组合并进行优化...")

# 遍历所有可能的无人机3机组合
for selected_indices in combinations(range(total_uav_count), uav_selection_count):
    selected_indices = list(selected_indices)

    # 1. 初始化种群
    population, pos_min, pos_max = initialize_population(WOA_POPULATION_SIZE, uav_selection_count)
    fitness_values = np.array([fitness_function_with_individual(p, selected_indices)[0] for p in population])
    best_idx = np.argmin(fitness_values)
    best_position = population[best_idx]
    best_fitness = fitness_values[best_idx]

    # 2. 执行鲸鱼优化算法 (WOA)
    for iter in range(WOA_ITERATIONS):
        # 衰减因子 a 从2线性衰减到0
        decay_factor = 2 * (1 - iter / WOA_ITERATIONS)
        population = woa_update_population(population, best_position, decay_factor, pos_min, pos_max)

        fitness_values = np.array([fitness_function_with_individual(p, selected_indices)[0] for p in population])
        min_idx = np.argmin(fitness_values)
        if fitness_values[min_idx] < best_fitness:
            best_fitness = fitness_values[min_idx]
            best_position = population[min_idx]

    # 3. 使用 SLSQP 进行局部精炼
    optimization_result = minimize(
        lambda x: fitness_function_with_individual(x, selected_indices)[0],
        best_position,
        method='SLSQP',
        bounds=np.stack((pos_min, pos_max), axis=1)
    )

    # 计算最终的覆盖能力
    total_coverage, individual_coverages = fitness_function_with_individual(optimization_result.x, selected_indices)

    result = {
        "uav_combination": [UAV_IDS[i] for i in selected_indices],
        "total_coverage": -total_coverage,
        "positions": [optimization_result.x[3*i:3*i+3] for i in range(uav_selection_count)],
        "individual_coverages": individual_coverages
    }
    optimization_results.append(result)
    print(f"完成组合 {result['uav_combination']} 的优化。")

# --- 输出优化结果 ---
print("\n" + "="*80)
print("      无人机小组最优配置与投放点分析")
print("="*80)

# 找到总覆盖能力最大的组合
best_overall_result = max(optimization_results, key=lambda x: x["total_coverage"])

# 打印最佳组合的详细信息

```

```

print("\n 最佳无人机小组配置:")
print(f" > 小组成员: {' '.join(best_overall_result['uav_combination'])}")
print(f" > 总覆盖能力: {best_overall_result['total_coverage']:.4f}")

print("\n 最佳小组各无人机投放点与贡献:")
for uav_id, pos, cov in zip(best_overall_result["uav_combination"], best_overall_result["positions"],
best_overall_result["individual_coverages"]):
    print(f" - {uav_id}:")
    print(f" - 最终投放点: x={pos[0]:.1f}, y={pos[1]:.1f}, z={pos[2]:.1f}")
    print(f" - 单机覆盖贡献: {cov:.4f}")

# 打印所有组合的简要对比
print("\n" + "-"*80)
print("所有无人机组合的总覆盖能力对比:")
print("-" * 80)
sorted_results = sorted(optimization_results, key=lambda x: x["total_coverage"], reverse=True)
for i, result in enumerate(sorted_results):
    rank_str = f"#{i+1}"
    combo_str = ", ".join(result['uav_combination'])
    coverage_str = f"{result['total_coverage']:.4f}"
    print(f" {rank_str:<4} | 组合: {combo_str:<18} | 总覆盖能力: {coverage_str}")

print("-"*80)

```

附录五：问题 5 代码求解

该脚本的目标是解决一个多无人机、多导弹的协同防御优化问题。
通过两阶段混合优化方法（WOA + SLSQP），为无人机找到最佳投放点，以最大化对所有来袭导弹的联合覆盖能力。

```

import numpy as np
import os
from scipy.optimize import minimize
import random

# --- 全局常量和配置 ---

# 无人机、导弹和目标圆柱体的参数
CYLINDER_CENTER = np.array([0.0, 200.0, 0.0]) # 真目标圆柱体的底面圆心
CYLINDER_RADIUS = 7.0 # 圆柱体半径
CYLINDER_HEIGHT = 10.0 # 圆柱体高度

# 三枚来袭导弹的初始配置
MISSILE_CONFIGS = [
    {"id": "M1", "initial_pos": np.array([20000.0, 0.0, 2000.0]), "speed": 300.0},
    {"id": "M2", "initial_pos": np.array([19000.0, 600.0, 2100.0]), "speed": 300.0},
    {"id": "M3", "initial_pos": np.array([18000.0, -600.0, 1900.0]), "speed": 300.0},
]

# 五架无人机的初始位置
UAV_INITIAL_POSITIONS = {
    "UAV1": np.array([17800.0, 0.0, 1800.0]),
    "UAV2": np.array([12000.0, 1400.0, 1400.0]),
    "UAV3": np.array([6000.0, -3000.0, 700.0]),
    "UAV4": np.array([11000.0, 2000.0, 1800.0]),
    "UAV5": np.array([13000.0, -2000.0, 1300.0]),
}
UAV_IDS = list(UAV_INITIAL_POSITIONS.keys())

# 烟雾和无人机移动参数
SMOKE_RADIUS = 10.0 # 烟雾有效半径 (m)
MAX_COVERAGE_DURATION = 20.0 # 烟雾最大遮挡时长 (s)
MAX_UAV_DISTANCE = 10000.0 # 无人机最大可移动距离 (m)

# 鲸鱼优化算法（WOA）参数
WOA_POPULATION_SIZE = 60 # 种群大小
WOA_ITERATIONS = 400 # 迭代次数
RANDOM_SEED = 42 # 随机种子，确保结果可复现
random.seed(RANDOM_SEED)

```

```

np.random.seed(RANDOM_SEED)

# 输出目录设置
OUTPUT_DIRECTORY = "uav_smoke_outputs_optimized"
os.makedirs(OUTPUT_DIRECTORY, exist_ok=True)

# 无人机对不同导弹的基础覆盖能力矩阵
# 行代表无人机 (UAV1-UAV5)，列代表导弹 (M1-M3)
COVERAGE_MATRIX = np.array([
    [4.6, 7.686, 1.246],
    [3.08, 3.9, 1.746],
    [2.92, 2.471, 3.1],
    [3.501, 1.212, 0.989],
    [1.877, 3.215, 0.36]
])

# --- 核心功能：适应度函数 ---

def compute_coverage(uav_index, missile_index, uav_position, missile_position):
    """
    计算单个无人机对单个导弹的覆盖贡献。
    覆盖能力随距离指数衰减。
    """
    distance = np.linalg.norm(uav_position - missile_position)
    return COVERAGE_MATRIX[uav_index, missile_index] * np.exp(-distance / 5000.0)

def fitness_function(positions):
    """
    计算所有无人机对所有导弹的总覆盖能力。
    这是优化的目标函数。
    positions 是一个包含所有无人机三维坐标的一维数组。
    """
    total_coverage = 0.0
    for uav_idx, uav_id in enumerate(UAV_IDS):
        # 从一维数组中提取当前无人机的三维坐标
        uav_pos = positions[3 * uav_idx:3 * uav_idx + 3]

        # 计算无人机与其初始位置的距离，用于惩罚超出移动范围的情况
        dist_from_initial = np.linalg.norm(uav_pos - UAV_INITIAL_POSITIONS[uav_id])

        # 限制无人机的最大移动距离，如果超出则按比例减少覆盖贡献
        scale = min(1.0, MAX_UAV_DISTANCE / (dist_from_initial + 1e-6))

        # 遍历所有导弹，计算并累加覆盖贡献
        for m_idx, missile in enumerate(MISSILE_CONFIGS):
            coverage = compute_coverage(uav_idx, m_idx, uav_pos, missile['initial_pos'])
            total_coverage += coverage * scale

    # 优化算法寻找最小值，因此返回总覆盖能力的负值
    return -total_coverage

# --- 优化器函数 ---

def initialize_woa_population(pop_size):
    """
    初始化鲸鱼优化算法 (WOA) 的种群。
    每个个体 (鲸鱼) 代表一组无人机投放点。
    """
    # 定义每个无人机在三维空间中的搜索范围
    pos_min = np.array([pos[dim] - MAX_UAV_DISTANCE for pos in UAV_INITIAL_POSITIONS.values() for dim in range(3)])
    pos_max = np.array([pos[dim] + MAX_UAV_DISTANCE for pos in UAV_INITIAL_POSITIONS.values() for dim in range(3)])

    # 在搜索范围内随机生成初始种群
    return np.random.uniform(low=pos_min, high=pos_max, size=(pop_size, len(pos_min)))

def woa_update_population(population, best_position, decay_factor):
    """
    根据鲸鱼优化算法的规则更新种群位置。
    """

```


模拟鲸鱼向最优解移动的行为。

```
"""
new_population = np.zeros_like(population)
for i in range(population.shape[0]):
    r1, r2 = np.random.rand(), np.random.rand()
    A = 2 * decay_factor * r1 - decay_factor
    C = 2 * r2
    D = np.abs(C * best_position - population[i])
    new_population[i] = best_position - A * D

    # 确保更新后的位置仍在合法范围内
    new_population[i] = np.minimum(np.maximum(new_population[i], POSITION_MIN), POSITION_MAX)
return new_population

# --- 主程序入口 ---
if __name__ == "__main__":

    # 构建所有无人机位置的边界数组
    POSITION_MIN = np.array([UAV_INITIAL_POSITIONS[name][dim] - MAX_UAV_DISTANCE for name in
                             UAV_IDS for dim in range(3)])
    POSITION_MAX = np.array([UAV_INITIAL_POSITIONS[name][dim] + MAX_UAV_DISTANCE for name in
                             UAV_IDS for dim in range(3)])

    print("--- 多无人机协同防御优化开始 ---")

    # 1. 鲸鱼优化算法 (WOA) 全局搜索
    print("阶段一：使用鲸鱼优化算法进行全局搜索...")
    population = initialize_woa_population(WOA_POPULATION_SIZE)
    fitness_values = np.array([fitness_function(p) for p in population])
    best_index = np.argmin(fitness_values)
    best_position = population[best_index]
    best_fitness = fitness_values[best_index]

    for iteration in range(WOA_ITERATIONS):
        # 衰减因子 a 从 2 线性衰减到 0，控制探索与开发平衡
        decay_factor = 2 * (1 - iteration / WOA_ITERATIONS)
        population = woa_update_population(population, best_position, decay_factor)
        fitness_values = np.array([fitness_function(p) for p in population])
        min_index = np.argmin(fitness_values)
        if fitness_values[min_index] < best_fitness:
            best_fitness = fitness_values[min_index]
            best_position = population[min_index]
        if (iteration + 1) % 50 == 0:
            print(f" 迭代 {iteration + 1}/{WOA_ITERATIONS}：当前最佳总覆盖能力: {-best_fitness:.4f}")

    print("全局搜索完成。")

    # 2. 序列二次规划 (SLSQP) 局部精炼
    print("\n 阶段二：使用 SLSQP 进行局部精炼...")
    optimization_result = minimize(
        fitness_function,
        best_position,
        method='SLSQP',
        bounds=np.stack((POSITION_MIN, POSITION_MAX), axis=1)
    )
    best_final_position = optimization_result.x
    final_coverage = -fitness_function(best_final_position)

    print(f"优化终止信息: {optimization_result.message}")
    print("局部精炼完成。")

    # --- 结果展示 ---
    print("\n" + "=" * 60)
    print("      无人机最优投放点方案")
    print("=" * 60)

    print(f"> 最终总覆盖能力: {final_coverage:.4f}")

    print("\n > 各无人机最优投放点:")
    for uav_idx, uav_id in enumerate(UAV_IDS):
```



```

x, y, z = best_final_position[3 * uav_idx:3 * uav_idx + 3]
print(f"    - {uav_idx}: (x={x:.1f}m, y={y:.1f}m, z={z:.1f}m)")

print("\n" + "=" * 60)
print("  该方案旨在通过优化无人机编队位置，最大化对来袭导弹的联合防御效果。")
print("=" * 60)

```

附录六：问题 5 价值系数矩阵热力图绘制

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import matplotlib as mpl
from scipy.spatial.distance import cdist

# --- 全局配置与参数 ---

# 绘图配置，支持中文显示和正确处理负号
mpl.rcParams['font.sans-serif'] = ['SimHei']
mpl.rcParams['axes.unicode_minus'] = False

# 无人机和导弹的初始位置信息
MISSILE_POSITIONS = {
    "Missile1": np.array([20000.0, 0.0, 2000.0]),
    "Missile2": np.array([19000.0, 600.0, 2100.0]),
    "Missile3": np.array([18000.0, -600.0, 1900.0])
}

UAV_INITIAL_POSITIONS = {
    "UAV1": np.array([17800.0, 0.0, 1800.0]),
    "UAV2": np.array([12000.0, 1400.0, 1400.0]),
    "UAV3": np.array([6000.0, -3000.0, 700.0]),
    "UAV4": np.array([11000.0, 2000.0, 1800.0]),
    "UAV5": np.array([13000.0, -2000.0, 1300.0])
}

# 模型超参数
MAX_UAV_DISTANCE = 10000.0    # 无人机最大可移动距离
COST_WEIGHT = 0.2             # 部署成本权重
REDUNDANCY_WEIGHT = 0.5       # 冗余惩罚权重
DISTANCE_SCALE = 1000.0       # 距离缩放系数，用于计算冗余惩罚

# 无人机对导弹的基础覆盖能力矩阵
# 行：无人机 (UAV1-5)，列：导弹 (M1-3)
BASE_COVERAGE_MATRIX = np.array([
    [4.6, 7.686, 1.246],
    [3.08, 3.9, 1.746],
    [2.92, 2.471, 3.1],
    [3.501, 1.212, 0.989],
    [1.877, 3.215, 0.36]
])

UAV_IDS = list(UAV_INITIAL_POSITIONS.keys())
MISSILE_IDS = list(MISSILE_POSITIONS.keys())

# --- 数据计算逻辑 ---
def calculate_metrics():
    """计算部署成本、冗余惩罚和综合价值矩阵。"""
    # 计算部署成本（基于初始位置的距离）
    deployment_costs = np.zeros((len(UAV_IDS), len(MISSILE_IDS)))
    for i, uav_id in enumerate(UAV_IDS):
        for j, missile_id in enumerate(MISSILE_IDS):
            deployment_costs[i, j] = np.linalg.norm(UAV_INITIAL_POSITIONS[uav_id] -
                MISSILE_POSITIONS[missile_id])

    # 计算假设的最优投放点（用于评估冗余）
    optimal_target_points = {}
    for uav_id in UAV_IDS:

```

```

    optimal_target_points[uav_id] = {}
    for missile_id in MISSILE_IDS:
        optimal_target_points[uav_id][missile_id] = (UAV_INITIAL_POSITIONS[uav_id] +
MISSILE_POSITIONS[missile_id]) / 2

# 计算冗余惩罚
redundancy_penalties = np.zeros((len(UAV_IDS), len(MISSILE_IDS)))
for i, uav_id in enumerate(UAV_IDS):
    for j, missile_id in enumerate(MISSILE_IDS):
        current_pos = optimal_target_points[uav_id][missile_id]
        same_missile_dists = []
        for k, other_uav in enumerate(UAV_IDS):
            if k != i and BASE_COVERAGE_MATRIX[k, j] > 0:
                other_pos = optimal_target_points[other_uav][missile_id]
                dist = np.linalg.norm(current_pos - other_pos)
                same_missile_dists.append(dist)
        if not same_missile_dists:
            redundancy_penalties[i, j] = 0
        else:
            avg_dist = np.mean(same_missile_dists)
            redundancy_penalties[i, j] = 1.0 / (1.0 + avg_dist / DISTANCE_SCALE)

# 归一化所有矩阵
normalized_coverage = normalize_matrix(BASE_COVERAGE_MATRIX)
normalized_deployment_costs = normalize_matrix(deployment_costs)
normalized_redundancy_penalties = normalize_matrix(redundancy_penalties)

# 计算综合价值系数矩阵
value_matrix = normalized_coverage - COST_WEIGHT * normalized_deployment_costs - REDUNDANCY_WEIGHT
* normalized_redundancy_penalties

    return deployment_costs, redundancy_penalties, value_matrix

def normalize_matrix(matrix):
    """将矩阵数据归一化到 [0, 1] 范围。"""
    min_val = np.min(matrix)
    max_val = np.max(matrix)
    if max_val - min_val < 1e-10:
        return np.zeros_like(matrix)
    return (matrix - min_val) / (max_val - min_val)

# --- 主程序入口 ---
if __name__ == "__main__":

    # 1. 计算所有任务指标矩阵
    deployment_costs, redundancy_penalties, value_matrix = calculate_metrics()

    # 2. 将数据转换为 DataFrame 以便可视化和输出
    value_df = pd.DataFrame(value_matrix, index=UAV_IDS, columns=MISSILE_IDS)

    # 3. 打印所有矩阵数据
    print("--- 任务指标矩阵 ---")
    print("\n 基础遮蔽能力矩阵:")
    print(pd.DataFrame(BASE_COVERAGE_MATRIX, index=UAV_IDS, columns=MISSILE_IDS))
    print("\n 部署成本矩阵 (距离):")
    print(pd.DataFrame(deployment_costs, index=UAV_IDS, columns=MISSILE_IDS))
    print("\n 冗余惩罚矩阵:")
    print(pd.DataFrame(redundancy_penalties, index=UAV_IDS, columns=MISSILE_IDS))
    print("\n 综合价值系数矩阵:")
    print(value_df)

    # 4. 可视化综合价值系数矩阵
    plt.figure(figsize=(14, 10))
    vmax = np.max(np.abs(value_matrix))
    vmin = -vmax

    # 创建热力图
    im = plt.imshow(value_matrix, cmap='RdYlGn', interpolation='bilinear', aspect='auto', vmin=vmin, vmax=vmax)

    # 为每个单元格添加数值标注

```

```

for i in range(value_df.shape[0]):
    for j in range(value_df.shape[1]):
        value = value_df.iloc[i, j]
        text_color = 'black' if abs(value) < vmax * 0.6 else 'white'
        plt.text(j, i, f'{value:.2f}', ha='center', va='center', color=text_color, fontsize=10, fontweight='medium',
alpha=0.9)

# 设置图表标签和标题
plt.xticks(ticks=np.arange(len(value_df.columns)), labels=value_df.columns, fontsize=12)
plt.yticks(ticks=np.arange(len(value_df.index)), labels=value_df.index, fontsize=12)
plt.title('协同多因子任务分配(CMFTA)价值系数矩阵', fontsize=16, pad=20, fontweight='bold')
plt.xlabel('导弹目标', fontsize=14, labelpad=12)
plt.ylabel('无人机平台', fontsize=14, labelpad=12)

# 添加颜色条
cbar = plt.colorbar(im, pad=0.05)
cbar.set_label('综合价值系数', fontsize=12)
cbar.ax.tick_params(labelsize=10)

# 添加网格线
plt.grid(False)
for i in range(len(value_df.index) + 1):
    plt.axhline(i - 0.5, color='gray', linestyle='-', linewidth=0.5, alpha=0.3)
for j in range(len(value_df.columns) + 1):
    plt.axvline(j - 0.5, color='gray', linestyle='-', linewidth=0.5, alpha=0.3)

# 调整布局和添加注释
plt.subplots_adjust(bottom=0.15, right=0.85)
plt.figtext(
    0.98, 0.02,
    f'注: 价值系数  $V_{ij} = B_{ij} - \lambda_{cost} \cdot Cost_{ij} - \lambda_{redund} \cdot Redund_{ij}$ \n'
    f'超参数:  $\lambda_{cost} = \{COST\_WEIGHT\}$ ,  $\lambda_{redund} = \{REDUNDANCY\_WEIGHT\}$ \n'
    f'B_ij: 遮蔽时长 | Cost_ij: 部署距离成本 | Redund_ij: 冗余惩罚',
    ha='right', va='bottom', fontsize=10, style='italic',
    bbox={'facecolor': 'white', 'alpha': 0.7, 'pad': 5}
)

plt.tight_layout(pad=2.0)
plt.show()

```